

APLICAȚIE MOBILĂ DEDICATĂ SISTEMULUI SANITAR

Candidat: Mădălina-Teodora, UNGUREAN

Coordonator științific: Conf. dr. ing. Adriana, ALBU

Sesiunea: Iunie 2022

REZUMAT

În cadrul lucrării de față am să prezint pe larg detalii legate de proiectul dezvoltat. Aplicația mobilă implementată are ca scop atingerea unor funcționalități destul de simple, dar care pot fi considerate de bază pentru o cunoștință minimă în ceea ce privește diferite afecțiuni care pot apărea oricărei persoane.

Printre funcțiile puse la dispoziție de către aplicație, sunt cele care permit utilizatorului să obțină informații despre diferite medicamente prin scanarea codului de bare de pe cutie. O altă opțiune implementată este cea de a găsi informații de contact specifice cadrelor medicale sau de obținere a informațiilor despre diferite afecțiuni, acestea fiind împărțite pe zone ale corpului uman. De asemenea, această aplicație pune la dispoziția utilizatorului posibilitatea de a seta alarmă cu notițe pentru a nu rata un tratament sau îi permite acestuia să poarte un dialog cu un robot de discuții, cu scopul de a se relaxa.

Toate informațiile utilizate în cadrul aplicației au fost preluate și verificate cu ajutorul cărților de medicină dedicate studenților de la facultatea de medicină din afara țării, motiv pentru care o ultimă funcționalitate adusă aplicației este cea de alegere limba de afișare a textului din aplicație.

Cuprins

CAPITOLUL 1: INTRODUCERE	4
CAPITOLUL 1.1: CONTEXT	4
CAPITOLUL 1.2: TEMA PROPRIU-ZISĂ	5
CAPITOLUL 2: STUDIU BIBLIOGRAFIC	7
CAPITOLUL 2.1: APLICAȚII DESTINATE INFORMĂRII UTILIZATORILOR ÎN LEGĂTURĂ CU DIFERITE AFECȚIUNI ÎNTÂLNITE FRECVENT	7
CAPITOLUL 2.2: APLICAȚII ANDROID DE TIPUL CHATBOT INTELIGENT	8
CAPITOLUL 2.3: APLICAȚII MOBILE UTILIZATE PENTRU SCANAREA CODURILOR DE BARE	9
CAPITOLUL 2.4: COMPARAȚIE ÎNTRE APLICAȚIILE MENȚIONATE ANTERIOR ȘI APLICAȚIA DEZVOLTATĂ, MEDICALFRIEND	10
CAPITOLUL 3: FUNDAMENTARE TEORETICĂ	11
CAPITOLUL 3.1: ANDROID STUDIO	11
CAPITOLUL 3.2: LIMBAJUL DE PROGRAMARE JAVA	12
CAPITOLUL 3.3: LIMBAJUL XML	12
CAPITOLUL 3.4: PLATFORMA FIREBASE	13
CAPITOLUL 4: SPECIFICAȚIILE APLICAȚIEI.....	14
CAPITOLUL 4.1: SCHEMA BLOC A SISTEMULUI. SCURTĂ DESCRIERE A APLICAȚIEI 	14
CAPITOLUL 4.2: FUNCȚIILE SISTEMULUI.....	15
CAPITOLUL 4.3: INTERFAȚA CU UTILIZATORUL.....	16
CAPITOLUL 4.4: STRUCTURA BAZEI DE DATE	24
CAPITOLUL 5: PROIECTAREA DE DETALIU	26
CAPITOLUL 5.1: ARHITECTURA PROGRAMULUI	27
CAPITOLUL 5.2: PREZENTAREA CODULUI SPECIFIC FUNCȚIONALITĂȚILOR.....	28
CAPITOLUL 6: UTILIZAREA SISTEMULUI	35
CAPITOLUL 7: CONCLUZII.....	40
CAPITOLUL 8: BIBLIOGRAFIE.....	41
Listă de figuri	43
Listă de tabele	44

CAPITOLUL 1: INTRODUCERE

CAPITOLUL 1.1: CONTEXT

În urma perioadei de pandemie, s-a putut observa mult mai ușor gradul de necesitate al unui canal de comunicare sau al unui mediu în care anumite activități esențiale vieții de zi cu zi, se pot desfășura într-o manieră sigură. Ținând cont de faptul că diferite evenimente (precum implementarea unei situații de urgență provocată de o pandemie) ne pot afecta activitatea zilnică și ne pot limita în ceea ce privește modul de realizare al unor activități, care în mod normal, ne pot părea banale (precum activitățile didactice, activitățile gospodărești sau cele care țin de mediul medical), este necesară găsirea unei alternative cât mai practice.

Cele mai multe domenii de activitate au fost nevoite să mute operațiunile în mediul online sau chiar să stopeze acțiunile de producție. Un domeniu în care activitatea a fost condiționată este cel medical. În perioada de pandemie controlul medical, care nu presupunea o situație critică, se desfășura prin apelarea medicului de familie sau a medicului de gardă de către o asistentă de la poarta spitalului pentru a transmite simptomele, iar acest apel era însoțit de poze (dacă era necesar). Tot acest proces putea fi ușurat, dacă un mediu, care să cuprindă informații despre afecțiunile comune și care nu prezintă un grad ridicat de risc asupra stării de bine a pacienților (precum răceala comună, dureri abdominale, dureri de cap, durere de spate, etc.), sigur și demn de încredere, era pus la dispoziția tuturor persoanelor. Cu ajutorul acestei îmbunătățiri s-ar putea reduce dimensiunea fluxului de pacienți care stau o perioadă bună de timp în fața cabinetelor medicale și de asemenea am avea parte de persoane mai bine informate, care ar înțelege cauza și efectele afecțiunii specifice simptomelor, dar și efectul tratamentului prescris de medic.

Un lucru cert, în ziua de azi, este faptul că fiecare persoană a ajuns să utilizeze telefonul mobil sau calculatorul sau are în jurul său o persoană care o face. Fiind tehnologii așa de utilizate, diferite activități au ales crearea de platforme care să fie echivalente cu activitatea desfășurată în incinta firmelor, însă se încearcă evitarea contactului în mod direct între angajat și client, într-o măsură cât mai mare. Printre aceste domenii se numără serviciile de livrare a produselor la domiciliu, activitățile didactice, plata facturilor, activități bancare, etc., acestea au optat pentru asocierea diferitelor forme de servicii on-line.

În ceea ce privește platformele utilizate, un articol de pe internet [1] evidențiază faptul că oamenii optează pentru utilizarea telefoanelor inteligente în detrimentul computerelor personale din diverse motive, precum comoditatea sau ușurința de transportare. Ținând cont de această preferință a utilizatorilor, următoarea întrebare care apare în momentul în care cineva dorește să achiziționeze un astfel de dispozitiv este "Care sistem de operare este mai potrivit, Android sau iOS?". Întrebarea anterioară subliniază cele mai utilizate sisteme de operare care se găsesc pe piață în momentul actual.

În paragraful anterior am menționat termenul de sistem de operare, care poate fi înțeles în felul următor conform [2], totalitatea programelor care ajută utilizatorul să acceseze resursele puse la dispoziție de către sistem.

În ceea ce privește aplicațiile mobile de tip android, acestea au avantajul de a rula pe toate dispozitivele care prezintă ca și sistem de operare android, dar și pe sistemul de operare windows cu ajutorul diferitelor tipuri de emulatoare.

În această lucrare va fi implementată o aplicație care vine în ajutorul persoanelor care din cauza situației actuale nu pot ajunge în contact direct cu un medic, deoarece în urma unei consultații telefonice nu au fost introduse în categoria persoanelor cu nevoie urgentă de consultație în formă fizică. Aceasta este categoria de utilizatori care cu ajutorul acestui proiect, propus dezvoltării, se pot documenta sau chiar ar putea găsi un nou medic care are disponibilitatea de a oferi mai multe informații sau chiar un control.

CAPITOLUL 1.2: TEMA PROPRIU-ZISĂ

Aplicația android, "MedicalFriend", dezvoltată în cadrul acestui proiect este destinată domeniului medical. Aceasta dorește să atingă mai multe proprietăți precum informarea utilizatorului în legătură cu diferite afecțiuni, setarea de alarme pentru amintirea diferitelor tratamente sau crearea unui cont pentru utilizatori.

Funcționalitățile aplicației sunt împărțite în două mari categorii și anume proprietăți puse la dispoziția utilizatorilor care nu doresc crearea unui cont în cadrul aplicației și cei care doresc să se înregistreze în aplicație. În funcție de această separare fiecare categorie va prezenta funcționalități diferite.

Pentru toți utilizatorii, în momentul în care aplicația este accesată, sunt puse la dispoziție următoarele posibilități: crearea de cont de utilizator, cu date adăugate în baza de date specifică aplicației, dar și funcționalitatea de prezentarea a informațiilor specifice diferitelor afecțiuni. În ceea ce privește crearea de cont, fiecare persoană trebuie să introducă următoarele date, numele, prenumele, adresa de email și o parolă. Aceste date vor fi stocate în cadrul bazei de date, unde și un cod unic de utilizator va fi creat pentru fiecare intrare. Aceste date vor fi utilizate pentru autentificarea utilizatorilor în aplicație, dar și pentru a evita crearea mai multor conturi de utilizator cu aceleași date. Cea de-a doua funcționalitate, menționată la începutul acestui paragraf, cuprinde detalii despre diferite afecțiuni, des întâlnite, împărțite în funcție de diferite părți ale corpului uman.

De asemenea, pe lângă cele două proprietăți menționate anterior, în acest punct, utilizatorii pot alege și limba în care doresc să fie afișat textul aplicației. Persoanele pot alege între următoarele patru limbi, engleză, germană, română și franceză. Însă această proprietate vine cu anumite restricții în ceea ce privește unele interfețe specifice unor funcționalități, dar acest lucru am să îl menționez în paragrafele următoare.

În cadrul aplicației, două dintre cele patru limbi menționate anterior sunt utilizate mai frecvent, în ceea ce privește disponibilitatea lor de a afișa conținutul text al unei interfețe. Acestea sunt limba engleză și română.

De asemenea un utilizator nelogat în aplicație mai are accesul la o funcționalitate și anume scanarea unui cod de bare specific unui medicament. Scopul acestei proprietăți este de a înlocui clasicul prospect care se găsește în cutiile de medicamente. Necesitatea acestei implementări vine din situațiile pe care le putem întâlni zilnic precum deschiderea cutiilor de medicamente în cadrul farmaciilor pentru a pune la dispoziția clienților prospectul, la solicitarea

acestui. După cum am menționat în paragrafele anterioare, această funcționalitate face parte din categoria celor care limitează din punct de vedere lingvistic deoarece în baza de date au fost adăugate diferite coduri de bare specifice medicamentelor pe care le întâlnim la noi în țară, pentru care prospectul este în limba română.

În urma creării de cont și utilizarea credențialelor pentru logarea în aplicație, se va încărca o interfață care permite alegerea unei opțiuni dintre următoarele: vizualizarea unei liste de medici din țară cu diferite specializări, setarea unei alarme, deschiderea unui robot interactiv dedicat conversațiilor sau părăsirea contului de utilizator.

În ceea ce privește listarea medicilor, această funcționalitate are la bază ideea de a gestiona ușor și în timp util situațiile de urgență care necesită date de contact sau indicații legate de locația unui medic cu o anumită specialitate. În cadrul aplicației afișarea medicilor se realizează utilizând baza de date care stochează informațiile menționate anterior despre diferite cadre medicale. Pentru a surprinde mai multe date specifice fiecărei intrări a bazei de date corespunzătoare nodului dedicat medicilor s-a utilizat și proprietatea platformei de a stoca imagini (după cum poate fi observat în figura 1).



Figura 1-Spațiul utilizat în cadrul Firebase Storage pentru imaginile specifice informațiilor despre medici

Din baza de date, pentru această funcționalitate, se preiau datele următoare: numele, specializarea și orașul specifice fiecărui medic. Cu ajutorul imaginilor se adaugă informații legate de adresă, orar, date de contact și numele clinicii în care medicul activează, dar și o poză cu acesta. Această funcționalitate este implementată pentru medici specifici țării noastre.

Pentru setarea unei alarme în cadrul aceste aplicații, utilizatorul trebuie doar să aleagă ora și data exactă, dar și să menționeze motivul pentru care setează această alarmă.

Ultima funcționalitate implementată în cadrul acestui proiect este cea de robot inteligent de discuții. Pentru această funcționalitate, în funcție de fiecare răspuns dat de către utilizator sau fiecare text introdus de către acesta, mai bine spus, se va genera un răspuns al chatbot-ului în așa fel încât conversația să aibă continuitate. Proprietatea fiind implementată cu ajutorul unui API online al unui chatbot aduce o limitare din punct de vedere al limbii utilizate, adică este utilizată doar limba engleză nu și celelalte menționate la începutul acestui subcapitol.

Totalitatea funcționalităților prezentate în această parte a documentației alcătuiesc aplicația mobilă denumită MedicalFriend, dedicată persoanelor obișnuite care din necesitate sau dorința de a-și îmbogăți cunoștințele caută o sursă de documentare.

CAPITOLUL 2: STUDIU BIBLIOGRAFIC

Influențele domeniului IT sunt puternic regăsite în majoritatea activităților desfășurate zilnic de către om sau pentru acesta (de exemplu: cumpărături, plata taxelor sau a facturilor, efectuarea diferitelor programări sau interacțiunea cu alte persoane prin intermediul mediului online, etc.), iar una dintre cele mai importante ramuri în care se utilizează informatica este medicina. Acest domeniu este în continuă dezvoltare, cu scopul de a reduce timpul de execuție și de a crește performanța diferitelor metode și tehnici utilizate în tratarea pacienților (aici se poate aminti de utilizarea inteligenței artificiale în dezvoltarea diferiților roboți implicați în efectuarea operațiilor, precum robotul Da Vinci sau diferite aparate utilizate în diagnosticarea sau tratarea cancerului, a afecțiunilor cardiace, stomatologice, etc.).

Chiar dacă în acest moment se dispune de aparatură performantă care ușurează și grăbește procesul de diagnosticare, dezvoltarea economică sau gradul scăzut de educație al diferitelor zone nu facilitează oferirea populației unor astfel de soluții.

Utilizarea unor tehnologii avansate din punctul de vedere al algoritmilor și programelor implementate presupune și o implicare mare din punctul de vedere al resurselor materiale, condiție care nu poate fi îndeplinită la fel de ușor la nivel global.

Pentru a pune în evidență zonele în care se încadrează aplicația dezvoltată, în continuare am să descriu aplicațiile similare ca și funcționalități întâlnite în diferite articole științifice sau pe MagazinPlay.

CAPITOLUL 2.1: APLICAȚII DESTINATE INFORMĂRII UTILIZATORILOR ÎN LEGĂTURĂ CU DIFERITE AFECȚIUNI ÎNTÂLNITE FRECVENT

Un studiu realizat și utilizat în motivarea dezvoltării unei aplicații mobile, EasyDetectDisease, destinate informării și tratării diferitelor afecțiuni arată faptul că numărul copiilor care își pierd viața până la vârsta de 5 ani la nivel global, din cauza diferitelor afecțiuni este ridicat [3].

Acest studiu a pornit de la ideea menționată în paragrafele anterioare, slaba dezvoltare a statelor, dar și nivelul scăzut de informare al părinților în ceea ce privește afecțiunile care se dovedesc a fi fatale unui copil cu vârsta până în 5 ani. Soluția propusă de autorii articolului a fost implementarea unei aplicații android în domeniul sănătății, cu scopul de a educa părinții despre posibile afecțiuni și simptome astfel încât acestea să poată fi prevenite, iar procentul de răspândire al acestor afecțiuni întâlnite în rândul copiilor sub cinci ani să scadă [3]. Pe parcursul studiului și al implementării, cei trei autori au insistat pe dezvoltarea funcționalităților următoare: detalierea informațiilor despre diferitele boli infecțioase și al simptomelor corespunzătoare, cauzele acestor afecțiuni, perioada de incubare, măsurile de prevenție corespunzătoare, informații în ceea ce privește alimentația recomandată, videoclipuri în care sunt prezentați copii în poziția pacienților sau în care apar medici pediatri sau experți în sănătate care promovează sănătatea în rândul copiilor. De asemenea această aplicație pune la dispoziția utilizatorului informații în ceea ce privește diagnosticarea bolilor infecțioase, bazată pe diferitele simptome identificate [3]. Această aplicație a fost dată spre testare în Pakistan, o țară aflată în zona slab

dezvoltate. În ceea ce privește dezvoltarea aplicației EasyDetectDisease (interfața poate fi observată în figura 2), informațiile utilizate pentru procesul de diagnosticare au fost colectate din diferite surse și evaluate de cadre medicale competente, iar apoi adăugate aplicației finale.

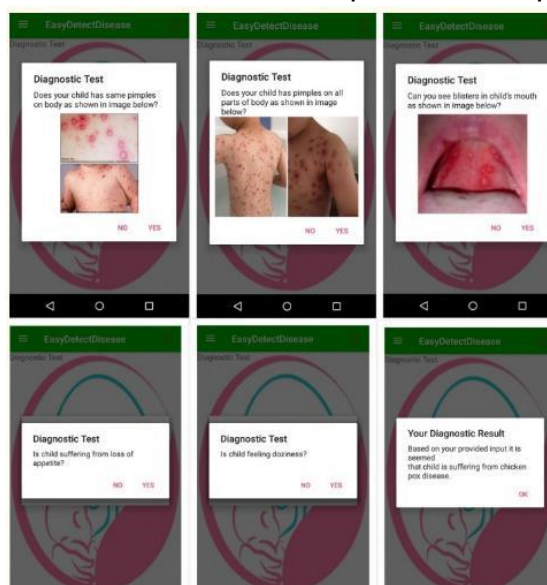


Figura 2-Interfața aplicației de informare
EasyDetectDisease

CAPITOLUL 2.2: APLICAȚII ANDROID DE TIPUL CHATBOT INTELIGENT

Cu peste cinci milioane de descărcări și o notă de 4.7/5 stele în Play Store, aplicația dezvoltată de Ada Health, are ca scop diagnosticarea utilizatorului în urma unei "conversații" purtate cu chatbot-ul dezvoltat cu ajutorul inteligenței artificiale.

În AppStore aplicația este cotate cu 4.9/5 stele, iar într-un clasament al aplicațiilor încadrate la categoria medicină, aceasta este pe locul 17, însă această clasare nu cuprinde același tip de aplicații iOS.

Cea mai frecventă întrebare care apare legat de acest tip de aplicație este "De ce am avea încredere că o aplicație axată pe sănătate, care utilizează inteligența artificială, poate oferi suport pentru utilizatori reali?". Din acest punct de vedere, în studiu se regăsește faptul că în spatele Ada-check your health, se află stocate între 8000-150000 de simptome și constatare, 15000-120000 de condiții și posibile boli. Algoritmul se bazează pe comparația (testată) dintre diagnostice reale și rezultate oferite de Ada [4]. Pentru a verifica nivelul calității acestei dezvoltări s-au realizat testări la nivel de unitate, prin interacțiune cu utilizatorul sau bazate pe cazuri prezentate în cărțile de specialitate. În ceea ce privește testarea de unitate, inteligența artificială a răspuns corect în recunoașterea celor mai evidente boli, pe când în cazul testării folosind un utilizator, aplicația a reușit să identifice diagnosticul corect, chiar dacă unele simptome au fost diferite [4].

Interfața cu utilizatorul este simplă (după cum se observă în figura 3), iar fiecare alegere făcută în cadrul aplicației ajută la ghidarea utilizatorului spre următorii pași. Aplicația este disponibilă în peste 130 de țări, iar utilizatorul poate alege din șapte posibile opțiuni pentru limba în care aplicația este utilizată, printre care și limba română [4]. Chatbot-ul compară răspunsurile

primite din partea utilizatorului cu simptomele diferitelor afecțiuni înregistrate în baza de date, pentru a determina următoarele întrebări astfel încât numărul de posibile diagnostice să fie minimizat, însă în momentul în care tiparul prezentat de utilizator nu mai coincide cu diagnosticele selectate de aplicație, numărul acestora va crește pentru a reidentifica majoritatea afecțiunilor posibile. Numărul de întrebări depinde de cât de comune sunt simptomele prezentate de utilizator, astfel o diagnosticare realizată de Ada poate dura câteva secunde sau câteva minute.

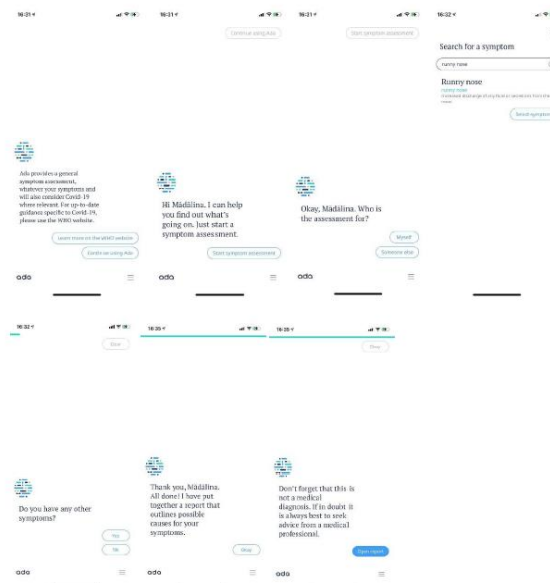


Figura 3-Interfața aplicației de tip chatbot Ada

CAPITOLUL 2.3: APLICAȚII MOBILE UTILIZATE PENTRU SCANAREA CODURILOR DE BARE

Pe lângă aceste două aplicații relevante pentru tema descrisă la punctul unu din acest referat, cel de-al doilea tip de implementare android relevantă, este cea care presupune dezvoltarea unei funcționalități de scanare a codului de bare. Această posibilitate are la bază dorința de a deține informațiile relevante, sau mai bine spus, cele mai importante (precum dozaj, efecte adverse, ingrediente) pentru diferite categorii de medicație, la doar o atingere distanță în orice moment al zilei sau în orice locație, fără ca utilizatorul aplicației să fie nevoit să caute prospectul.

În partea de început a studiului este evidențiat faptul că ideea principală de la care a pornit acest proiect este legată de evitarea pierderii timpului în momentul în care se dorește realizarea plății produselor într-un magazin, de asemenea prin această aplicație se dorește și evitarea formării de mulțimi la cozile de așteptare [5]. Principiul de funcționare al aplicației ShopEZY, descris pe parcursul articolului științific, este următorul: un utilizator al aplicației utilizează ShopEZY, în cadrul unui magazin, pentru a scana codul de bare al produselor, moment în care toate detaliile produsului sunt afișate pe ecranul telefonului mobil, acestea urmează să fie transmise spre server-ul magazinului cu ajutorul conexiuni wi-fi. Fiecare produs scanat trece prin acest proces, apoi este adăugat la lista cu produse scanate formată în

aplicație. Pasul final presupune transformarea listei formate în cadrul aplicației, într-un bon fiscal și transmiterea acestuia în sistemul magazinului, pentru ca apoi utilizatorul să poată achita cumpărăturile și să poată pleca [5].

CAPITOLUL 2.4: COMPARAȚIE ÎNTRE APLICAȚIILE MENȚIONATE ANTERIOR ȘI APLICAȚIA DEZVOLTATĂ, MEDICALFRIEND

În tabelul 1 se pot observa cele mai importante funcționalități întâlnite în aplicațiile prezentate anterior, dar și cele urmărite în dezvoltarea aplicației MedicalFriend.

Caracteristici	Ada	EasyDetectDisease	ShopEZY	MedicalFriend
Chatbot	X			X
Informații despre afecțiuni	X	X		X
Scanare cod de bare			X	X
Videoclipuri interactive		X		
Achiziție în aplicație	X			
Alarmă				X
Listă cu informații despre cadre medicale				X
Informații despre alimentația corectă		X		

Tabel 1- Tabel cu comparațiile între aplicațiile prezentate și aplicația dezvoltată

CAPITOLUL 3: FUNDAMENTARE TEORETICĂ

În acest capitol am să prezint bazele teoretice ale tehnologiilor utilizate pentru dezvoltarea aplicației specifice proiectului.

Codul specific funcționalităților, dar și cel al interfețelor cu utilizatorul au fost create cu ajutorul mediului de programare Android Studio în limbajul Java și XML (Extensible Markup Language), iar datele au fost stocate cu ajutorul platformei Firebase.

CAPITOLUL 3.1: ANDROID STUDIO

Android este un sistem de operare fundamentat pe nucleu Linux care permite dezvoltatorilor să folosească ca și limbaj de programare Java, Kotlin sau C++ și pune la dispoziția acestora biblioteci dezvoltate de Google.

Android studio este mediul de dezvoltare integrat, oficial dedicat dezvoltării de aplicații mobile de tipul android, bazat pe IntelliJ IDEA. Acest mediu de programare oferă funcționalități care ușurează procesul de creare al aplicațiilor, precum: un sistem de construcție flexibil bazat pe Gradle, un emulator rapid și bogat în funcții, un mediu unificat în care se pot dezvolta aplicații pentru toate dispozitivele Android, șabloane de cod și integrarea GitHub pentru a ajuta la crearea de funcții comune ale aplicației și importarea de cod exemplu, instrumente și cadre de testare extinse, instrumente Lint pentru a detecta performanța, gradul de utilizare, compatibilitatea versiunilor și alte probleme sau suport încorporat pentru Google Cloud Platform, facilitând integrarea Google Cloud Messaging și App Engine [6].

Structura unui proiect Android conține unul sau mai multe module cu fișiere de cod sursă și fișiere de resurse. Aceste module pot face parte din următoarele categorii: modulul de aplicație, modulul bibliotecii sau modulul Google App Engine. În mod implicit, Android Studio afișează fișierele de proiect în vizualizarea proiectului Android, așa cum se arată în figura 4. Această vizualizare este organizată pe module pentru a oferi acces rapid la fișierele sursă cheie ale proiectului [6].

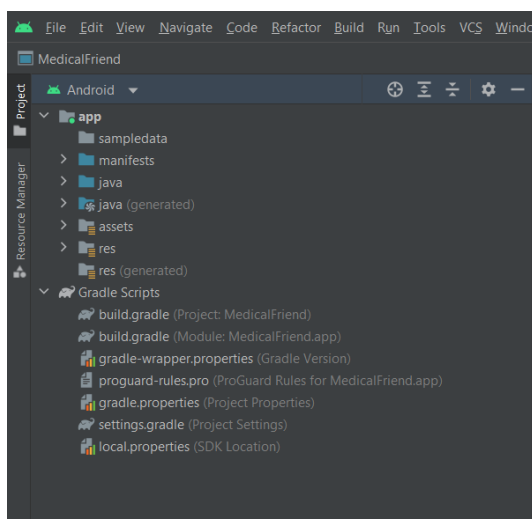


Figura 4-Organizarea pe module a aplicației MedicalFriend

Toate fișierele de compilare sunt vizibile la nivelul superior sub Gradle Scripts și fiecare modul de aplicație conține următoarele foldere: manifest (acesta conține fișierul AndroidManifest.xml), java (c onține fișierele codului sursă Java, inclusiv codul de testare JUnit) și res (conține toate resursele non-cod, cum ar fi aspecte XML, șiruri de interfață și imagini bitmap) [6].

CAPITOLUL 3.2: LIMBAJUL DE PROGRAMARE JAVA

Limbajul de programare Java este un limbaj de uz general, concurent, bazat pe clase, orientat pe obiecte. Este conceput pentru a fi suficient de simplu încât mulți programatori pot atinge fluentă în limbaj. Limbajul de programare Java este legat de C și C++, dar este organizat destul de diferit, cu o serie de aspecte ale C și C++ omise și câteva idei din alte limbaje incluse. Se dorește să fie un limbaj de producție, nu un limbaj de cercetare și astfel, așa cum a sugerat C. A. R. Hoare în lucrarea sa clasică despre designul limbajului, designul a evitat să includă caracteristici noi și netestate [7].

Limbajul de programare orientat pe obiecte, Java, a fost conceput de către James Gosling la începutul anilor 90, dar lansat în anul 1995 [8].

După cum am menționat și în primul paragraf al acestui capitol, acest limbaj presupune eliminarea unor idei, întâlnite în alte limbaje de programare, precum: moștenire multiplă, supraîncărcarea operatorilor sau a pointerilor [9]. O altă particularitate a limbajului dedusă din [9] este faptul că Java este independent de mașina pe care acesta rulează.

Din [10] aflăm faptul că în Java orice linie de cod trebuie să fie scrisă în interiorul unei clase pentru a rula. O altă caracteristică importantă a acestui limbaj de programare este faptul că acesta este case-sensitive, iar două cuvinte care diferă doar printr-o literă (adică dacă aceasta este mică sau mare) sunt considerate ca fiind două cuvinte distincte.

În ceea ce privește SDK-urile din Android Studio, acestea conțin bibliotecile standard din Java, precum bibliotecile specifice operațiilor matematice, specifice graficelor sau structurilor de date [11].

CAPITOLUL 3.3: LIMBAJUL XML

Limbajul XML este un limbaj de marcare (markup language) precum HTML, însă spre diferență de acesta, XML nu prezintă etichete (tag-uri) predefinite, ci acestea sunt definite în funcție de necesitățile dezvoltatorului care utilizează limbajul. Există multe limbaje bazate pe XML, inclusiv XHTML, MathML, SVG, RSS și RDF. De asemenea, un utilizator poate defini propriul său limbaj derivat din XML [12].

XML (Extensible Markup Language) este folosit pentru a descrie datele. Standardul XML este o modalitate flexibilă de a crea formate de informații și de a partaja electronic date structurate prin internetul public, precum și prin rețelele corporative [13].

XML funcționează oferind un format de date previzibil. XML este strict la formatare; dacă formatarea este dezactivată, programele care procesează sau afișează datele codificate vor returna o eroare. Pentru ca un document XML să fie considerat bine format -- adică conform cu

sintaxa XML și capabil să fie citit și înțeles de un parser XML -- trebuie să fie cod XML valid. Toate documentele XML constau din elemente; un element acționează ca un container pentru date. Începutul și sfârșitul unui element sunt identificate prin etichete de deschidere și închidere, cu alte elemente sau date simple în interior [14] (după cum se poate observa în figura 5).

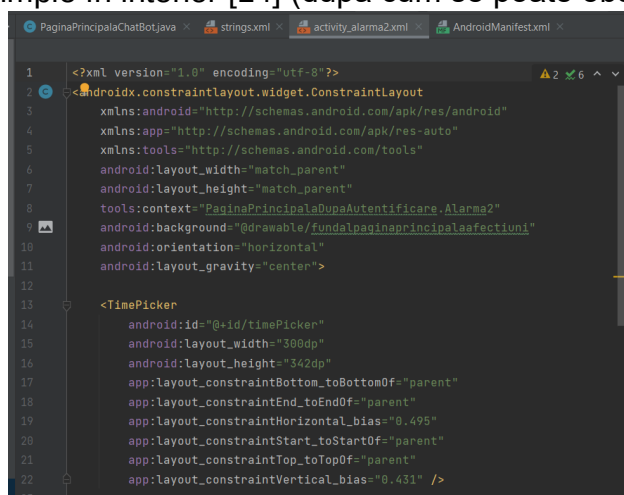


Figura 5-Exemplu de definire a unui fișier XML din cadrul proiectului MedicalFriend

CAPITOLUL 3.4: PLATFORMA FIREBASE

În cadrul acestui proiect, platforma Firebase a fost utilizată pentru îndeplinirea a trei funcționalități: definirea modului de autentificare a utilizatorilor, stocarea diferitelor date și stocarea imaginilor.

Platforma este pusă la dispoziția utilizatorilor de către Google. Firebase a apărut în anul 2011, fiind un proiect produs de către Firebase Inc., aceasta este o platformă pentru dezvoltarea aplicațiilor mobile și web, în 2014 platforma a fost achiziționată de către Google [15]. Firebase este o platformă care pune la dispoziția utilizatorului diferite unelte pentru a asigura suport pentru un proces de calitate superioară cu scopul de a ușura monitorizarea proiectului, pentru a putea acorda o atenție ridicată utilizatorilor. În momentul de față Firebase pune la dispoziția utilizatorului 20 de produse, acestea fiind împărțite în trei categorii: dezvoltarea aplicației (Cloud Firestore, ML Kit, Cloud Functions, Authentication, Hosting, Cloud Storage, Realtime Database), îmbunătățirea calității aplicației (Crashlytics, App Distribution, Performance Monitoring, Test Lab) și creșterea afacerii (Analytics, Extensions, Predictions, Firebase A/B Testing, Cloud Messaging, In-App Messaging, Remote Config, Dynamic Links, App Indexing) [16].

CAPITOLUL 4: SPECIFICAȚIILE APLICAȚIEI

CAPITOLUL 4.1: SCHEMA BLOC A SISTEMULUI. SCURTĂ DESCRIERE A APLICAȚIEI

În figura 6 poate fi observată schema bloc a aplicației MedicalFriend.

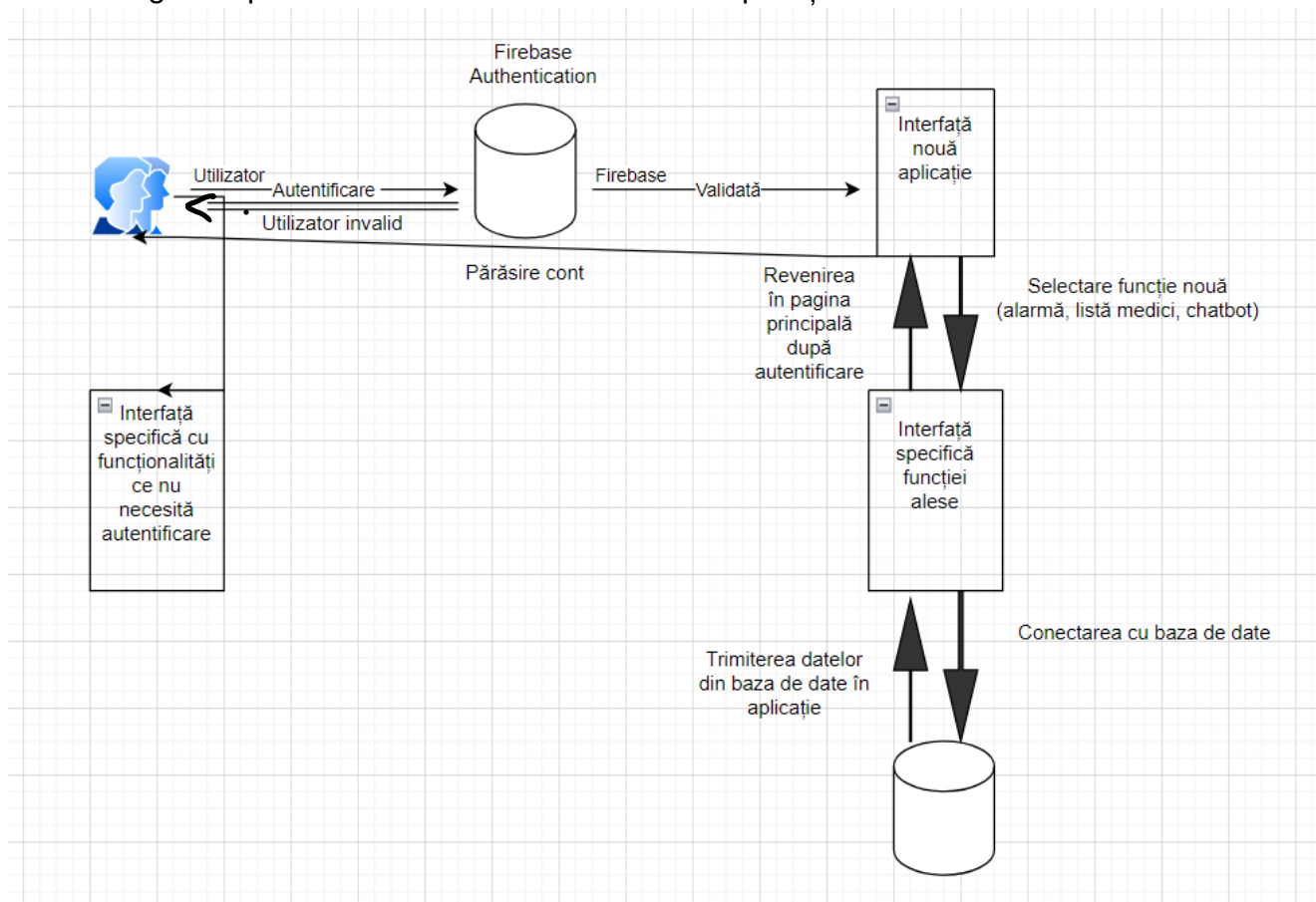


Figura 6-Schema bloc a aplicației dezvoltate

Conform schemei din figura 6, aplicația poate fi descrisă în următorul mod: în urma lansării în acțiune a aplicației de către un utilizator, în interfața principală acesta are posibilitatea de a explora funcționalități care nu necesită un cont, precum schimbarea limbii, vizualizarea de informații despre diferite afecțiuni sau scanarea codului de bare. Cea de-a doua posibilitate pusă la dispoziția utilizatorului este de a crea un cont, iar cu datele înregistrate în baza de date și verificarea acestora cu serverul Firebase, acesta poate accesa și restul funcționalităților puse la dispoziție de către aplicație, dacă datele introduse la autentificare sunt validate. După ce acești pași sunt parcurși o nouă interfață se va deschide, în cadrul căreia vor fi afișate celelalte proprietăți ale aplicației (vizualizarea unei liste cu diferiți medici, setarea unui memento, posibilitatea de a purta o discuție cu un robot inteligent sau părăsirea contului). Pentru aceste funcționalități se va face legătură cu baza de date pentru a extrage diferite informații stocate în cadrul acesteia, iar acestea vor fi transmise înapoi în cadrul aplicației, în interfețele specifice

fiecărei opțiuni. Dacă utilizatorul alege să părăsească contul, atunci aplicația va fi redirecționată spre pagina principală afișată în momentul în care, aceasta este lansată în acțiune, iar procesul descris în acest paragraf poate fi reluat de ori câte ori se dorește.

CAPITOLUL 4.2: FUNCȚIILE SISTEMULUI

Funcțiile sistemului dezvoltat sunt următoarele:

- Crearea unui cont de utilizator: în cadrul acestei funcționalități, **utilizatorul are posibilitatea de a înregistra datele sale, precum email, nume, parolă, pe platforma Firebase** cu scopul de a le salva și utiliza mai târziu pentru a avea acces la toate funcțiile aplicației;
- Autentificarea în contul de utilizator: scopul acestei funcții este **de a valida datele introduse de către utilizator și găsirea acestora în cadrul bazei de date**. Această funcționalitate are scopul de a crește gradul de siguranță pus la dispoziție de către aplicație;
- Schimbarea limbii: aceasta funcție, după cum îi spune și denumirea, **are rolul de a schimba limba de afișare a aplicației**. Dacă un utilizator dorește să arate informații unor persoane care nu înțeleg limba română, atunci acesta poate să aleagă afișarea în limba engleză, germană sau franceză.
- Listarea informațiilor despre afecțiuni: cu ajutorul acestei funcționalități **utilizatorul poate să caute într-o listă, pentru o parte a corpului specifică, diferite afecțiuni**, pe lângă afișarea posibilelor cauze, se mai pot vizualiza posibile analize, tratamente sau părerea medicilor de specialitate legată de subiectul respectiv;
- Scanarea codului de bare: dacă utilizatorul utilizează această funcție, atunci **la dispoziția acestuia sunt puse prospectele medicamentelor scanate**. În momentul în care un cod este scanat, este căutat în baza de date și informațiile sunt afișate în cadrul aplicației;
- Listarea medicilor: în cadrul acestei funcționalități, **se afișează informații despre cadre medicale din diferite sectoare ale ramurii medicinei** (precum cardiologie, ginecologie, ecografie 2D, etc). Printre informații se regăsesc numele, specializarea, orarul și locația de activare a medicului;
- Setarea alarmei: această funcție a fost implementată în ideea în care este mai ușor să se **gestioneze toate informațiile necesare unui memento** setat de către utilizator într-un singur loc;
- Chatbot: această funcționalitate are rolul de a oferi utilizatorului un moment de relaxare sau detașare de la problemele întâlnite zilnic, prin **interacțiunea directă cu un robot inteligent de conversații**, care dezvoltă conversația în funcție de răspunsurile oferite de utilizator;
- Leșire din cont: cu această funcție, **un utilizator poate alege să fie deconectat din cont** și astfel acesta poate să revină la funcțiile principale oferite de aplicație și chiar să ofere telefonul unei alte persoane pentru a căuta informațiile puse la

dispoziție de către aplicație, fără ca aceasta să aibă acces la funcționalitățile puse la dispoziție de către un cont de utilizator.

CAPITOLUL 4.3: INTERFAȚA CU UTILIZATORUL

În cadrul acestui subcapitol am să prezint design-ul interfețelor cu utilizatorul, dar am să și explic elementele utilizate pentru crearea lor, atât din punct de vedere teoretic, cât și utilizarea lor exactă în acest proiect.

- Interfața după lansarea în activitate a aplicației: pentru realizarea design-ului acestei interfețe, am utilizat două elemente specifice fișierelor XML folosite în Android Studio, `ConstraintLayout` și `ProgressBar`. Definiția oferită de către documentația pusă la dispoziție dezvoltatorilor de aplicații android pentru `ConstraintLayout` este următoarea: permite crearea unor layout-uri mari și complexe cu o ierarhie de vizualizare plată (fără grupuri de vizualizare imbricate). Toate vizualizările sunt așezate în funcție de relațiile dintre vizualizările fratelui și aspectul părinte [17]. În cadrul acestui tag am stabilit proprietăți precum înălțimea și lățimea pe care o va avea interfața în comparație cu dimensiunile ecranului (`android:layout_width="match_parent"`, `android:layout_height="match_parent"`), am atribuit o imagine de fundal pentru interfață (`android:background="@drawable/first"`) și de asemenea am definit clasa Java de care aparține fișierul (`tools:context=".MainActivity"`) (după cum se poate observa și în figura 7). În ceea ce privește `ProgressBar`-ul, acesta este definit în felul următor: un element de interfață cu utilizatorul care indică progresul unei operațiuni. Bara de progres acceptă două moduri pentru a reprezenta progresul: determinat și nedeterminat. Afișajul barei de progres unui utilizator într-un mod neîntreruptiv [18]. În cadrul aplicației, ca și proprietăți ale acestei etichete am utilizat stabilirea poziției în interfață a obiectului, design-ul pentru bara de progres, dar și id-ul utilizat în codul java pentru atribuirea unei funcționalități acestui obiect. Aceste caracteristici definite pot fi observate în figura 8.



Figura 8-Interfața specifică deschiderii aplicației

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    android:background="@drawable/first"
    >

    <ProgressBar
        android:id="@+id/progressBar"
        style="@android:attr/progressBarStyle"
        android:layout_width="346dp"
        android:layout_height="71dp"
        android:indeterminateDrawable="@drawable/cpb_1"
        android:indeterminateDuration="@android:integer/config_longAnimTime"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.492"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.633" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Figura 7-Captură de ecran cu activity_main.xml

Înainte de a continua acest capitol, am să menționez faptul că în toate interfețele am utilizat eticheta de tip `ConstraintLayout` deoarece aceasta oferă o încadrare mai bună pe diferite tipuri de ecrane ale dispozitivelor. Cu acest lucru fiind menționat mai departe nu am să mai amintesc la fiecare interfață acest tip de obiect utilizat.

De asemenea pentru toate obiectele din fișierle XML sunt proprietăți care nu pot lipsi din definirea acestora, precum cele de poziționare în cadrul interfeței, dar și cele de stabilire a dimensiunii obiectelor sau stabilirea unui id specific. Pentru a evita repetarea acestora la fiecare subpunct, explicația este aceeași oferită anterior, dar anumite caracteristici vor fi surprinse în imaginile ce or să fie adăugate pentru fiecare interfață ce urmează.

- Interfața principală a aplicației (se poate observa în figura 9): pentru construirea acestei interfețe am utilizat pe lângă obiectele deja descrise `Button` și `TextView`. Descrierea teoretică a obiectului `Button` este următoarea: un element de interfață cu utilizatorul pe care utilizatorul îl poate atinge sau face click pentru a efectua o acțiune [19]. O proprietate importantă pentru acest element este cel de stabilire al fundalului cu ajutorul căruia am atribuit un fișier XML în cadrul căruia am stabilit culoarea și forma butonului. De asemenea prin `android:text` am atribuit din fișierele care conțin string-urile aplicației textul specific de afișare pe buton. `TextView`-ul este un element de interfață cu utilizatorul cu ajutorul căruia se afișează un text utilizatorului. Aceste caracteristici pot fi observate în figura 10.



Figura 9-Interfața principală a aplicației

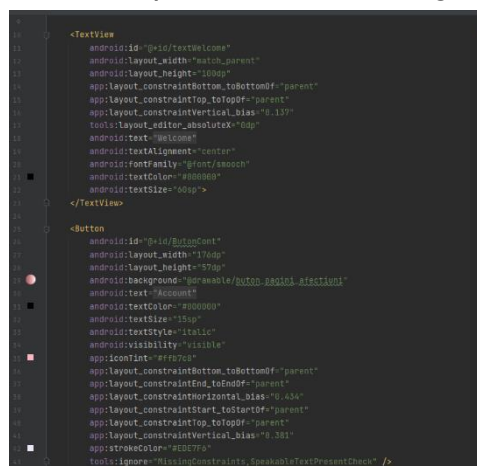


Figura 10-Captură de ecran cu codul XML specific obiectelor descrise la acest subpunct

- Interfața de autentificare a utilizatorului în cont (figura 12): pentru realizarea unui design complet care să coincidă unei funcționalități corecte a funcției, am utilizat elemente de tip `RelativeLayout` și `AutoCompleteTextView` (exemplificat în figura 11). Descrierea `RelativeLayout`-ului este următoarea: `RelativeLayout` este un grup de vizualizări care afișează vizualizările copil în poziții relative. Poziția fiecărei vizualizări poate fi specificată ca fiind relativă la elementele frate (cum ar fi la stânga sau dedesubtul unei alte vizualizări) sau în poziții relativ la zona parinte `RelativeLayout` (cum ar fi aliniată la partea de jos, la stânga sau la centru) [20]. Am utilizat acest element pentru a alinia elementele interfeței unele față de

celelalte, nu față de întreaga interfață. Conform [21] aflăm că o vizualizare text editabilă afișează automat sugestii de finalizare în timp ce utilizatorul tastează. Lista de sugestii este afișată într-un meniu derulant din care utilizatorul poate alege un element cu care să înlocuiască conținutul casetei de editare. În ceea ce privește proprietățile folosite pentru acest element, unul mai aparte este cel de afișare a unui text sugestiv pe post de indiciu în fiecare casetă înainte ca utilizatorul să scrie ceva în ea.

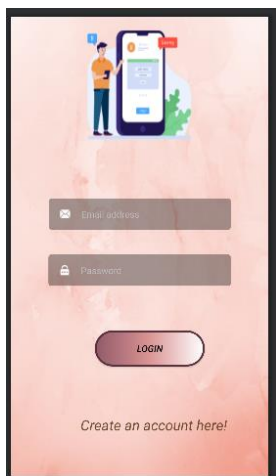


Figura 12-Interfața de autentificare a aplicației

```

10 <RelativeLayout
11     android:layout_width="match_parent"
12     android:layout_height="match_parent"
13     >
14     <AutoCompleteTextView
15         android:id="@+id/emailAutentificare"
16         android:layout_width="298dp"
17         android:layout_height="58dp"
18         android:layout_alignParentStart="true"
19         android:layout_alignParentTop="true"
20         android:layout_alignParentEnd="true"
21         android:layout_alignParentRight="true"
22         android:layout_marginStart="61dp"
23         android:layout_marginTop="289dp"
24         android:layout_marginEnd="59dp"
25         android:layout_marginRight="59dp"
26         android:background="@drawable/rounded_rect"
27         android:drawableStart="@drawable/icon_mail"
28         android:drawableLeft="@drawable/icon_mail"
29         android:drawablePadding="10dp"
30         android:hint="Email address"
31         android:inputType="textEmailAddress"
32         android:paddingStart="10dp"
33         android:paddingEnd="10dp"
34         android:textColor="@color/white"
35         android:textColorHint="@color/bcbcbc"
36         android:textSize="15sp" />

```

Figura 11-Exemplu de implementare a AutoCompleteTextView-ului din cadrul proiectului

- Interfața pentru crearea unui cont de utilizator: în cadrul acestei interfețe am utilizat doar elementele de tip obiect descrise până în acest moment, lucru care poate fi observat în figura 13 (design-ul interfeței) .

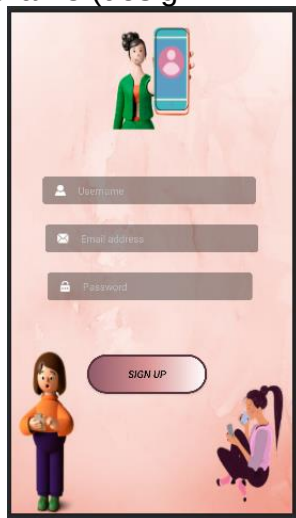


Figura 13- Interfața specifică funcției de creare cont pentru un utilizator

- Interfața de selectare a limbii pentru aplicație (figura 15): în cadrul acestei interfețe a fost utilizat un nou element și anume LinearLayout (exemplu în figura

14). Conform documentației android aflăm faptul că acest obiect are următorul rol, un aspect care aranjează alte vederi fie orizontal într-o singură coloană, fie vertical pe un singur rând [22].

O proprietate importantă utilizată pentru acest element este cea de a oferi posibilitatea de a atribui o funcționalitate la apăsarea pe acest element pe lângă poziționarea elementelor cuprinse în cadrul acestui obiect (*android:onClick="onClick"*). Această proprietate poate fi observată în figura 15.



Figura 15- Interfața de schimbare a limbii

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     xmlns:tools="http://schemas.android.com/tools"
6     android:layout_width="match_parent"
7     android:layout_height="match_parent"
8     tools:context=".SelectApplicationLanguage"
9     android:background="@drawable/fundallimba">
10
11     <LinearLayout
12         android:id="@+id/LimbaRomana"
13         android:layout_width="178dp"
14         android:layout_height="143dp"
15         android:onClick="onClick"
16         android:orientation="vertical"
17         app:layout_constraintBottom_toBottomOf="parent"
18         app:layout_constraintEnd_toEndOf="parent"
19         app:layout_constraintHorizontal_bias="0.107"
20         app:layout_constraintStart_toStartOf="parent"
21         app:layout_constraintTop_toTopOf="parent"
22         app:layout_constraintVertical_bias="0.056">
```

Figura 14-Exemplu de implementare a LinearLayout-ului din fișierele XML ale proiectului

- Interfața specifică unui utilizator oaspete: această interfață este una simplă, care conține doar elemente de tip ConstraintLayout și Button, cu ajutorul cărora utilizatorul este redirecționat spre paginile dorite. Design-ul acesteia poate fi observat în figura 16.

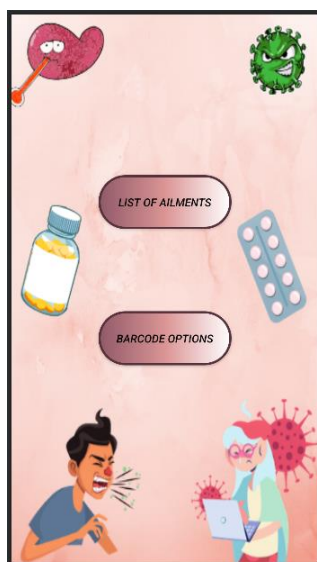


Figura 16-Interfața dedicată tuturor utilizatorilor fără necesitatea de a crea cont

- Interfața de vizualizare a listei de afecțiuni (figura 17): în cadrul acestui design intervine un nou element, Spinner (implementare în figura 18). Definiția acestui obiect este următoarea: spinner-urile oferă o modalitate rapidă de a selecta o valoare dintr-un set. În starea implicită, un spinner-ul arată valoarea sa selectată în prezent. Atingerea spinner-ului afișează un meniu derulant cu toate celelalte valori disponibile, din care utilizatorul poate selecta una nouă [23]. Pentru acesta, în codul XML, am setat doar proprietățile specifice id-ului, dimensiunilor și poziționării în cadrul aplicației, deoarece lista de valori și comportamentul acestuia au fost determinate cu ajutorul codului Java. Design-ul și codul XML pot fi vizualizate în figurile de mai jos.



Figura 18-Lista de valori afișate în urma apăsării Spinner-ului

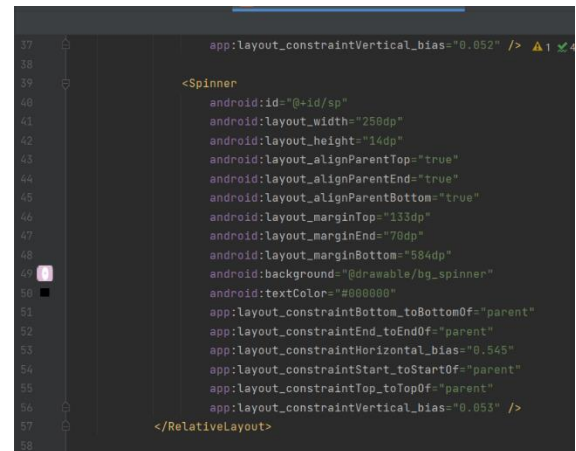


Figura 17-Codul XML specific Spinner-ului

- Interfața pentru scanarea codului de bare (figura 20): design-ul acestei pagini este completat de obiectul SurfaceView (implementare în figura 19), a cărei funcționalitate este următoarea: oferă o suprafață de desen dedicată încorporată în interiorul unei ierarhii de vizualizare. Puteți controla formatul acestei suprafețe și, dacă doriți, dimensiunea acesteia; SurfaceView are grijă să plaseze suprafața în locația corectă de pe ecran, conform [24]. Acest element are rolul, în codul proiectului, de a oferi un spațiu în care se accesează camera dispozitivului și care afișează imaginile surprinse de aceasta, toate acestea sunt realizate în cadrul codului Java.



Figura 19-Interfața dedicată scanării codului de bare

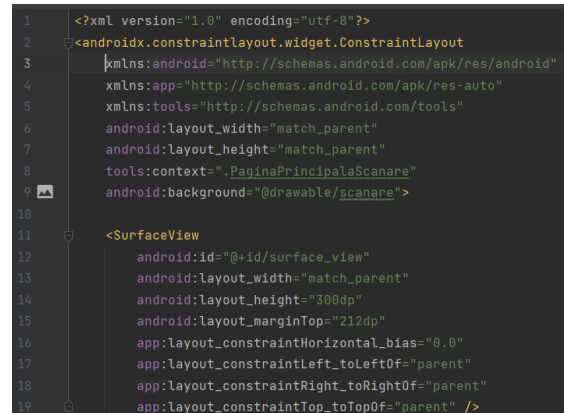


Figura 20-Codul XML pentru Spinner

- Interfața principală după autentificarea în cont (figura 22): este o interfață simplă, care conține elemente de tip buton, prin intermediul cărora se accesează interfețele specifice funcționalităților rămase (setare alarmă, afișare medici, chatbot).

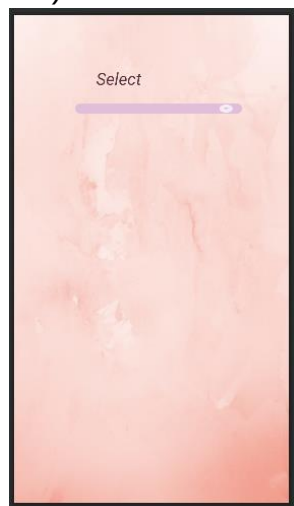


Figura 21-Interfața pentru alegerea afecțiunilor

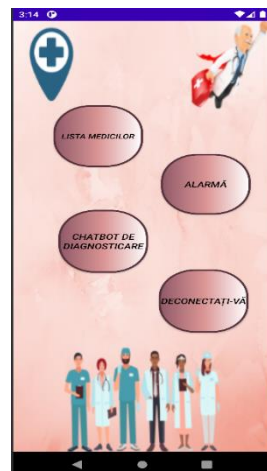


Figura 22-Interfața principală după autentificare

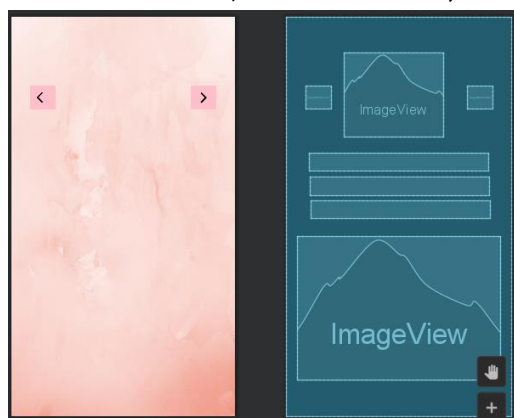
- Interfața pentru setarea alarmei (figura 23): ca și elemente noi utilizate în acest design sunt TimePicker și EditText. TimePicker este definit ca fiind un widget pentru selectarea orei din zi, fie în modul 24 de ore, fie în modul AM/PM [25], iar EditText ca fiind un element de interfață cu utilizatorul pentru introducerea și modificarea textului [26].



Figura 23-Interfața pentru setarea unei alarme

- Interfața de vizualizare a medicilor (figura 24 și 26): printre ultimele elemente introduse în fișierele XML pentru realizarea design-urilor interfețelor cu utilizatorul pentru această aplicație este ImageView-ul (implementare în figura 25). Acesta

are următorul rol: afișează resurse de imagine, de exemplu resurse Bitmap sau Drawable. ImageView este, de asemenea, folosit în mod obișnuit pentru a aplica nuanțe unei imagini și pentru a gestiona scalarea imaginii [27].



Figură 25-Design-ul din Android studio pentru interfața de listare a medicilor

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="PaginaPrincipalaDupaAutentificare.ListaMediciContacte"
    android:background="@drawable/fundalpaginaprincipalaafectiuni">
    <ImageView
        android:id="@+id/imageViewProfile"
        android:layout_width="153dp"
        android:layout_height="153dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.456"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.112"
        tools:ignore="MissingConstraints" />
```

Figura 24-Exemplu de cod XML pentru implementarea unui ImageView

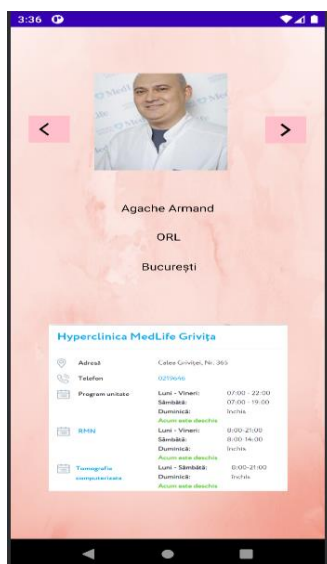


Figura 26-Exemplu de afișare a datelor despre medici din Firebase în momentul rulării aplicației

- Interfața de trecere între pagina principală afișată după autentificare și cea specifică chatbot-ului (figura 27): chiar dacă această interfață este una foarte simplă, am menționat-o deoarece pentru design-ul acesteia am utilizat un obiect nou, GifImageView. Rolul acestui element este de a permite afișarea pe interfața curentă a unei imagini animate a cărei extensie este .gif.

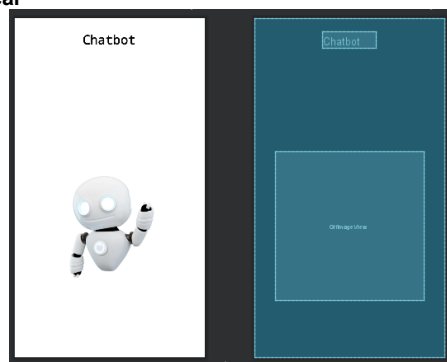


Figura 27-Interfața de tranziție, captură de ecran realizată în Android Studio

- Interfața dedicată chatbot-ului (figura 28): pentru a crea această interfață la elementele deja menționate se mai adaugă CardView (figura 31), utilizat pentru a construi spațiul dedicat răspunsului utilizatorului sau al robotului în cadrul interfeței (figura 29). Un penultim obiect utilizat este RecyclerView, acesta are rolul de a afișa sub formă de listă răspunsurile celor doi participanți în discuție (figura 30), iar un ultim element utilizat în fișierle XML este ScrollView a cărui funcționalitate este de a oferi utilizatorului posibilitatea de a purta o discuție mai lungă cu robotul și de a păstra toate replicile conversației cu posibilitatea de a naviga printre ele prin efectuarea de mișcări în sus sau jos cu degetul pe ecran. Fiecare dintre elementele menționate la acest subpunct au următoarele definiții: CardView este un widget nou în Android care poate fi folosit pentru a afișa orice fel de date, oferind un aspect de colț rotunjit împreună cu o anumită cotă. CardView este vizualizarea care poate afișa vizualizări una peste alta. Principala utilizare a CardView este că ajută la oferirea unei senzații și aspect bogat designului UI [28], RecyclerView afișează seturi mari de date în interfața de utilizare, minimizând în același timp utilizarea memoriei [29], iar ScrollView este un grup de vizualizări care permite derularea ierarhiei de vizualizări plasate în el. Vizualizarea derulare poate avea un singur copil direct plasat în ea. Pentru a adăuga mai multe vizualizări în vizualizarea de defilare, faceți copilul direct pe care îl adăugați un grup de vizualizare, de exemplu LinearLayout, și plasați vizualizări suplimentare în acel LinearLayout [30].

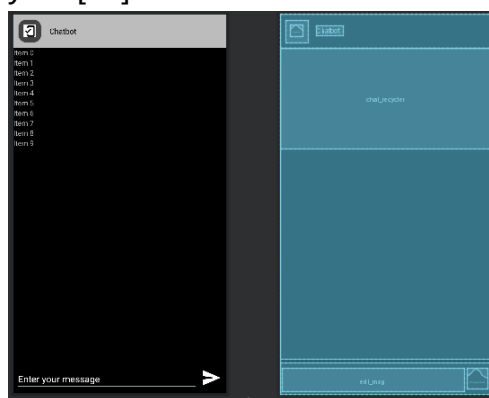


Figura 28-Interfață principală chatbot în Android Studio

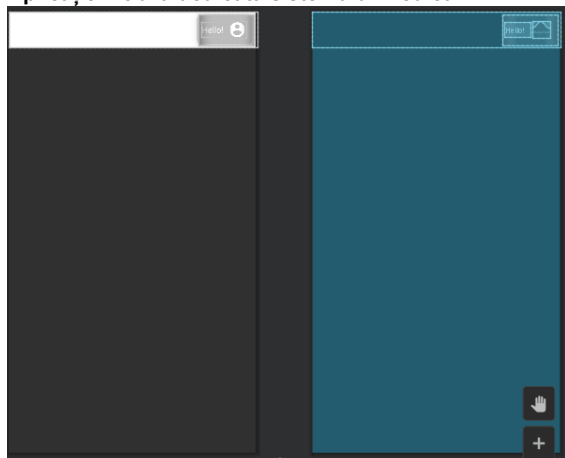


Figura 31-Imagine din Android Studio cu design-ul care utilizează CardView pentru crearea răspunsului utilizatorului

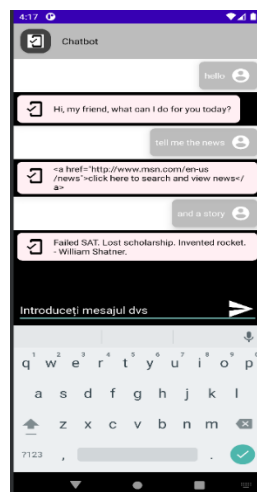


Figura 30-Imagine exemplificativă cu funcționalitatea chatbot-ului

```

10 <androidx.cardview.widget.CardView
11     android:layout_width="wrap_content"
12     android:layout_height="wrap_content"
13     android:layout_marginTop="5dp"
14     android:layout_marginEnd="5dp"
15     android:layout_marginBottom="5dp"
16     app:cardBackgroundColor="#000000"
17     app:cardCornerRadius="5dp"
18     app:cardElevation="10dp"
19     app:layout_constraintBottom_toBottomOf="parent"
20     app:layout_constraintEnd_toEndOf="parent"
21     app:layout_constraintTop_toTopOf="parent">
22     <LinearLayout
23         android:layout_width="wrap_content"
24         android:layout_height="wrap_content"
25         android:orientation="horizontal">
26         <TextView
27             android:layout_width="wrap_content"
28             android:layout_height="wrap_content"
29             android:layout_gravity="center"
30             android:layout_margin="5dp"
31             android:padding="10dp"
32             text="Hello!"
33             android:id="@+id/userMsg"
34             android:textColor="@color/white" />

```

Figura 29-Codul XML pentru crearea design-ului bulinei de răspuns a utilizatorului cu ajutorul CardView

CAPITOLUL 4.4: STRUCTURA BAZEI DE DATE

Baza de date utilizată pentru acest proiect este oferită de către platforma online Firebase. Această bază de date este una non-relațională, ci este una bazată pe structura de graf cu noduri.

Baza de date creată pentru acest proiect a fost legată prin intermediul platformei la aplicația dezvoltată în Android Studio. Structura acesteia poate fi observată în figura 32.



Figura 32- Structura bazei de date

În imaginea de mai sus se poate observa faptul că baza de date conține trei noduri principale, care conțin informații necesare pentru diferite funcționalități ale aplicației în felul următor: nodul Medici conține informații despre cadrele medicale care sunt afișate în interfața de listare a acestora, nodul Pastile conține informațiile specifice pentru diferite tipuri de medicamente pe care le întâlnim în prospectul acestora, iar în nodul Users sunt salvate datele specifice fiecărui cont de utilizator.

- Nodul Medici (din figura 33) are următoarea structură: pentru mai mulți indexi (cuprinși între 1 și 15) sunt salvate informațiile nume, oraș și specializare, care sunt extrase și afișate în interfață.

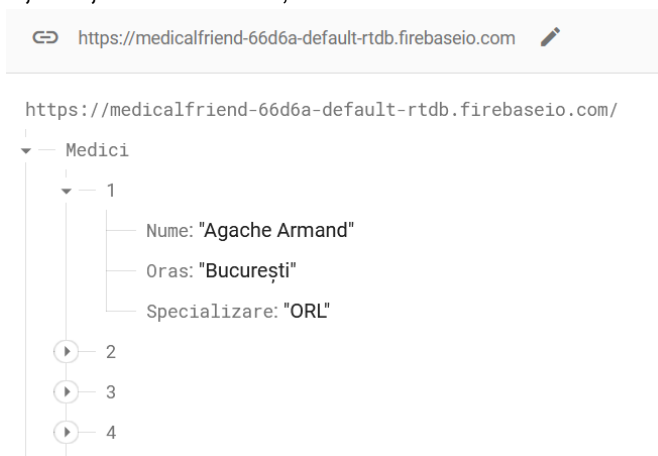


Figura 33- Structura nodului Medici

- Structura nodului Pastile (din figura 34): fiecare nod copil al acestui nod rădăcină are ca și denumire cifrele codului de bare ale diferitelor medicamente, iar la rândul său acesta conține informații precum compoziția, contraindicațiile, indicațiile și precauțiile specifice medicamentului respectiv.

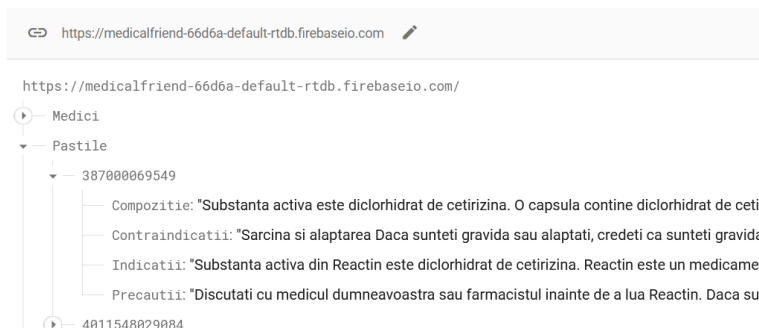
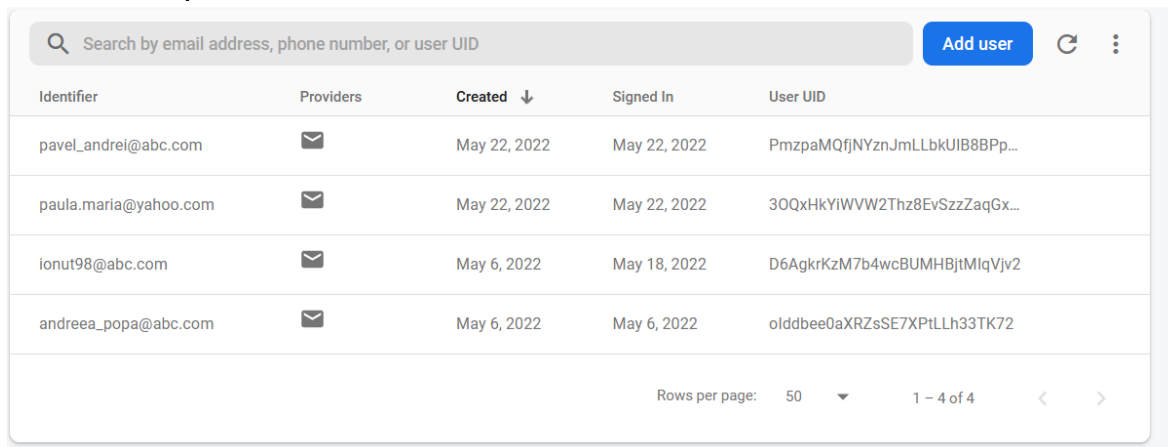


Figura 34-Structura nodului Pastile

- Structura nodului Users (din figura 37): ultimul nod component al bazei de date, are ca și noduri copii informațiile despre fiecare utilizator înregistrat. Fiecare nod are ca și denumire indicele unic de identificare (uid), atribuit fiecărui utilizator în momentul creării de cont. Acesta poate fi observat în partea de Firebase Authentication (figura 35). Pentru fiecare nod copil sunt salvate date precum numele de utilizator, adresa de email utilizată pentru crearea contului și parola.

Din motive de siguranță parola fiecărui utilizator este afișată sub formă următoare
*****.



Identifier	Providers	Created ↓	Signed In	User UID
pavel_andrei@abc.com	✉	May 22, 2022	May 22, 2022	PmzpaMQfjNYznJmLLbkUIB8BPp...
paula.maria@yahoo.com	✉	May 22, 2022	May 22, 2022	30QxHkYiWVW2Thz8EvSzzZaqGx...
ionut98@abc.com	✉	May 6, 2022	May 18, 2022	D6AgkrKzM7b4wcBUMHBjtMIqVjv2
andreea_popa@abc.com	✉	May 6, 2022	May 6, 2022	oIddbee0aXRZsSE7XptLLh33TK72

Figura 35-Informațiile salvate în Firebase Authentication la crearea unui cont de utilizator

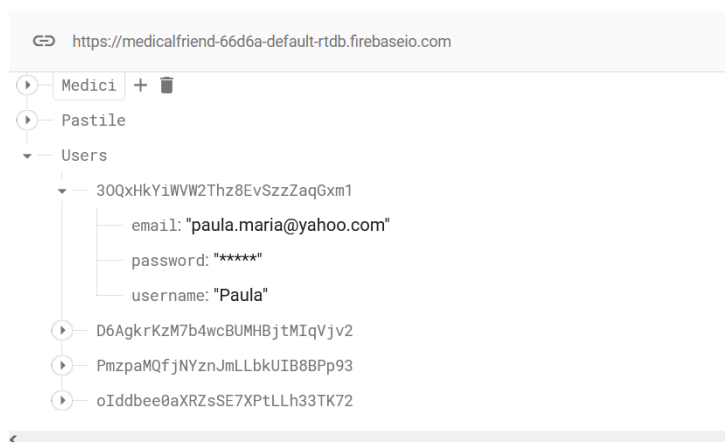
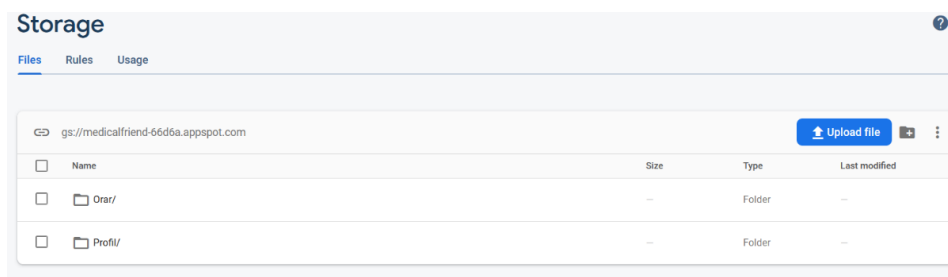


Figura 37-Structura nodului Users

În cadrul proiectului pe lângă Realtime Database pus la dispoziție de către Firebase pentru stocarea datelor și Firebase Authentication am mai folosit și opțiunea de Storage a platformei pentru a putea stoca din imaginile utilizate în dezvoltarea aplicației. Structura acestuia este una destul de simplă, alcătuită din două foldere care conțin imagini specifice pentru două informații afișate în interfața de listare a medicilor (poza acestora de profil și poza cu orarul și direcțiile de localizare ale cabinetelor), fiecare poză având ca nume un index.



Name	Size	Type	Last modified
Orar/	—	Folder	—
Profil/	—	Folder	—

Figura 36-Structura Firebase Storage

CAPITOLUL 5: PROIECTAREA DE DETALIU

CAPITOLUL 5.1: ARHITECTURA PROGRAMULUI

Arhitectura sistemului dezvoltat poate fi observată în figura 38.

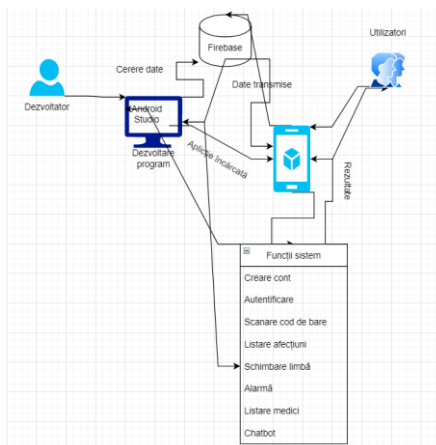


Figura 38- Schema arhitecturală a sistemului dezvoltat

În diagrama din figura 38 am încercat să surprind tehnologiile utilizate pentru dezvoltarea acestei aplicații (Android studio, cu limbajul Java, XML și Firebase, pentru stocarea de date și poze), dar și faptul că aplicația comunică cu serverul bazei de date atât cât este rulată în timpul dezvoltării în cadrul mediului de programare, dar și după ce aceasta este încărcată pe dispozitivul fizic.

Un alt aspect pe care am încercat să îl evidențiez prin această sechemă este faptul că dezvoltatorul/ utilizatorul la momentul utilizării aplicației fie ea în mediul de dezvoltare, fie încărcată pe dispozitiv are de asemenea conexiune cu baza de date. Un alt aspect important este că persoana care utilizează are acces la toate funcțiile sistemului în funcție de dorințele și necesitățile personale legate de aceasta (funcționalitățile disponibile în cadrul aplicației sunt surprinse în cadrul schemei).

Pentru ca aplicația să funcționeze trebuie să fie rulată pe una din următoarele variante, fie un dispozitiv cu sistem de operare Android, de preferat cu o versiune egală sau mai mare cu Oreo 8.0/ 8.1 pentru a avea acces la toate funcțiile sau un dispozitiv pe care se poate instala un Emulator Android, pe care trebuie de asemenea respectată condiția cu versiunea minimă de android instalată (listă disponibilă în figura 39).

Nume	Numărul versiunii	Data lansării stabilei inițiale	Suportat (fixări de securitate)	Nivel API
Fără nume de cod oficial	1.0	23 septembrie 2008	Nu	1
	1.1	9 februarie 2009	Nu	2
Cupcake	1.5	27 aprilie 2009	Nu	3
Donut	1.6	15 septembrie 2009	Nu	4
Eclair	2.0 - 2.1	26 octombrie 2009	Nu	5 - 7
Froyo	2.2 - 2.2.3	20 mai 2010	Nu	8
Gingerbread	2.3 - 2.3.7	6 decembrie 2010	Nu	9 - 10
Honeycomb	3.0 - 3.2.6	22 februarie 2011	Nu	11 - 13
Ice Cream Sandwich	4.0 - 4.0.4	18 octombrie 2011	Nu	14 - 15
Jelly Bean	4.1 - 4.3.1	9 iulie 2012	Nu	16 - 18
KitKat	4.4 - 4.4.4	31 octombrie 2013	Nu	19 - 20
Lollipop	5.0 - 5.1.1	12 noiembrie 2014	Nu	21 - 22
Marshmallow	6.0 - 6.0.1	5 octombrie 2015	Nu	23
Nougat	7.0 - 7.1.2	22 august 2016	Nu	24 - 25
Oreo	8.0 - 8.1	21 august 2017	Da	26 - 27
Pie	9	6 august 2018	Da	28
Android 10	10	3 septembrie 2019	Da	29
Android 11	11	8 septembrie 2020	Da	30

Figura 39- Lista cu versiunile Android lansate, în ordine cronologică

CAPITOLUL 5.2: PREZENTAREA CODULUI SPECIFIC FUNCȚIONALITĂȚILOR

În acest subcapitol am să prezint cele mai semnificative părți ale codului Java specific implementării funcționalităților aplicației.

Înainte de a începe să detaliez funcțiile și clasele aș dori să menționez un aspect, pentru toate obiectele și elementele descrise în capitolul anterior s-au declarat variabile în cadrul claselor și s-a utilizat funcția de identificare a elementului specific cu ajutorul id-ului definit în codul XML pentru fiecare în parte. Am să exemplific acest cod odată pentru un tip de elemente, iar mai apoi nu am să mai repet această explicație la fiecare parte de cod prezentată. De exemplu pentru un element de tipul `ProgressBar` utilizat în fișierul XML, căruia i s-a atribuit id `progressBar`, în clasa java se va defini o variabilă de tipul `ProgressBar progressBar`; pentru ca mai apoi în funcția principală a clasei `onCreate` să se realizeze legătura între obiect și variabila cu ajutorul căreia o să atribui o funcționalitate în felul următor: `progressBar=findViewById(R.id.progressBar)`. După `R.id` se trece numele id-ului atribuit în codul XML pentru obiect.

- Prima funcție pe care aș dori să o prezint este cea de transfer între două interfețe după expirarea unui timp transmis ca parametru al funcției (figura 40). Această funcție am utilizat-o de mai multe ori pe parcursul proiectului pentru a schimba interfețele în cazul în care una dintre interfețe nu prezenta buton sau alte metode de realizare a transferului. Pentru că această funcție nu are nevoie de obiect anume definit în fișierul XML este necesară definirea în cadrul clasei a unei variabile de tipul `Timer` cu ajutorul căreia o să fie implementată o *programare* în urma căreia se stabilește ca primul parametru clasa care urmează să fie deschisă la consumarea timpului stabilit de cel de-al doilea parametru.

```
public class MainActivity extends AppCompatActivity {

    Timer timer;
    private View decorView;
    private ProgressBar progressBar;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        overridePendingTransition(0, 0);

        //pentru progress bar
        timer=new Timer();
        timer.schedule(0, 1) {
            Intent intent=new Intent(MainActivity.this, PaginaSelectareTipContUtilizator.class);
            startActivity(intent);
            finish();
        }, delay(2000));
    }
}
```

Figura 40- Codul funcției de transfer între două interfețe cu delay din clasa Java specifică activității principale a aplicației

- Deoarece prima activitate a aplicației este una simplă care prezintă doar denumirea acestora și o imagine de fundal, din punct de vedere estetic am ales înlăturarea barei de opțiuni din partea de jos a ecranului unui telefon mobil (după cum se poate observa în figura 41).

```
//pentru ascunderea barei din josul paginii
decorView.getWindow().getDecorView();
decorView.setOnSystemUiVisibilityChangeListener(new View.OnSystemUiVisibilityChangeListener() {
    @Override
    public void onSystemUiVisibilityChange(int visibility) {
        if(visibility==0)
            decorView.setSystemUiVisibility(hideSystemBars());
    }
});

private void init() { this.progressBar=findViewById(R.id.progressBar); }

//pentru action bar
@Override
public void onWindowFocusChanged(boolean hasFocus) { ... }
private int hideSystemBars() {
    return View.SYSTEM_UI_FLAG_LAYOUT_STABLE
        |View.SYSTEM_UI_FLAG_IMMERSIVE_STICKY
        |View.SYSTEM_UI_FLAG_LAYOUT_FULLSCREEN
        |View.SYSTEM_UI_FLAG_LAYOUT_HIDE_NAVIGATION
        |View.SYSTEM_UI_FLAG_FULLSCREEN
        |View.SYSTEM_UI_FLAG_HIDE_NAVIGATION;
}
```

Figura 41- Captură de ecran cu codul specific ascunderii barei de opțiuni

- O altă funcție întâlnită chiar de la deschiderea aplicației este cea pentru afișarea barei de progres (figura 42). În cadrul acestei funcții stabilind timpul de delay pe ecran va apărea o bară ciclică, care are rolul de a marca timpul rămas până la încărcarea noii interfețe. Dar înainte de a stabili cât timp va apărea aceasta în interfață trebuie setată vizibilitatea obiectului.

```
//pentru progress bar
progressBar.setVisibility(View.VISIBLE);
new Handler().postDelayed(new Runnable() {
    @Override
    public void run() { progressBar.setVisibility(View.GONE); }
}, delayMillis: 1900);
```

Figura 42-Cod java pentru implementarea funcționalității barei de progres

- O altă modalitate de a realiza o tranziție între activitatea curentă și o alta din cadrul proiectului este atribuirea funcționalității unui buton (figura 43). În momentul în care un buton este apăsat acesta să apeleze funcția *startactivity(Intent intent)* cu ajutorul căreia prin intermediul parametrului se stabilește interfața curentă și cea în care se dorește tranziția. Codul java poate fi observat în figura 43.

```
butonGuest.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        Intent int2=new Intent( packageContext: PaginaSelectareTipContUtilizator.this, GuestInterfataPrincipala.class);
        startActivity(int2);
    }
});
```

Figura 43-Exemplu de cod java care atribuie posibilitatea de a schimba activitățile prin apăsarea unui element de tip buton din cadrul interfeței curente

- Altă parte a codului care este importantă este cea prin care se realizează schimbarea limbii de afișare a aplicației. Pentru design-ul acestei interfețe am ales adăugarea unor elemente de tip `LinearLayout`, mai exact patru astfel de obiecte specifice fiecărei opțiuni. În codul java pentru cele patru variabile se apelează funcția specifică de apăsare (`setOnClickListener(view v)`, figura 44), în cadrul căreia se setează codul specific fiecărui fișier string (ro, fr, de sau en) pentru a ști ce traducere se utilizează, iar apoi cu aceste detalii stabilite se reconstruiește afișarea activității.

```
LinearLayout ro=findViewById(R.id.LimbaRomana);
LinearLayout de=findViewById(R.id.LimbaGermana);
LinearLayout fr=findViewById(R.id.LimbaFranceza);
LinearLayout en=findViewById(R.id.LimbaEngleza);
LanguageManager lang= new LanguageManager( ctx, this);
ro.setOnClickListener(view -> {
    lang.updateResources( code: "ro");
    recreate();
});
de.setOnClickListener(view -> {
    lang.updateResources( code: "de");
    recreate();
});
fr.setOnClickListener(view -> {
    lang.updateResources( code: "fr");
    recreate();
});
en.setOnClickListener(view -> {
    lang.updateResources( code: "en");
    recreate();
});
```

Figura 44-Funcția java de setare a limbii de afișare a aplicației

- Pentru interfața de autentificare s-a implementat următorul cod java pentru funcționalități, în partea incipientă am utilizat o funcție definită de către mine în clasa specifică activității cu ajutorul căreia am atribuit id-urile butonului, casetelor text în care se completează, un câmp text cu ajutorul căruia se realizează transferul spre activitatea de înregistrare, dar și instanța firebase specifică autentificării (după cum se poate observa în figura 45).

```
private void initializeGUI(){

    email = findViewById(R.id.emailAutentificare);
    parola = findViewById(R.id.parolaAutentificare);
    btnAutentificare = findViewById(R.id.ButonAutentificare);
    progressDialog = new ProgressDialog( context, this);

    firebaseAuth = FirebaseAuth.getInstance();

}
```

Figura 45-Funcția de inițializare din clasa de autentificare

După ce elementele au fost inițializate corespunzător, într-o variabilă de tipul `FirebaseUser` am salvat indicele de identificare unic al utilizatorului curent și am

făcut o simplă verificare ($!=null$) pentru a determina dacă în momentul curent este un user autentificat în aplicație. Dacă verificarea este adevărată atunci interfața de autentificare nu este afișată, iar utilizatorul este redirecționat la pagina următoare. În caz contrar se așteaptă ca user-ul să introducă datele necesare și să apese butonul pentru a apela funcția de validare a datelor introduse și înregistrarea lor, în cazul în care validarea este una de succes (după cum se poate observa în figura 46).

```
user = firebaseAuth.getCurrentUser();

if(user != null) {
    finish();
    startActivity(new Intent( packageContext: PaginaAutentificare.this, paginaPrincipalaDupaAutentificare.class));
}

btnAutentificare.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {

        String inEmail = email.getText().toString();
        String inPassword = parola.getText().toString();

        if(validateInput(inEmail, inPassword)){
            signUser(inEmail, inPassword);
        }
    }
});
```

Figura 46-Cod java al funcției de autentificare utilizator cu Firebase

În funcția de semnare a utilizatorului (figura 47) care are ca și parametrii două valori string care reprezintă adresa de email și parola introduse de către utilizator în câmpurile de autocompletare, se apelează pentru instanța de autentificare a firebase, definită anterior, funcția de înregistrare cu ajutorul parametrilor menționați. Dacă această activitate are un rezultat pozitiv, atunci datele sunt înregistrate pe platforma Firebase, iar activitatea următoare este deschisă. În caz contrar se afișează un mesaj de atenționare asupra faptului că adresa de email sau parola pot fi greșite.

```
public void signUser(String email, String password){

    progressDialog.setMessage("Verification...");
    progressDialog.show();

    firebaseAuth.signInWithEmailAndPassword(email, password).addOnCompleteListener(new OnCompleteListener<AuthResult>() {
        @Override
        public void onComplete(@NonNull Task<AuthResult> task) {
            if(task.isSuccessful()){
                progressDialog.dismiss();
                Toast.makeText( context: PaginaAutentificare.this, "Login successful!", Toast.LENGTH_SHORT).show();
                startActivity(new Intent( packageContext: PaginaAutentificare.this, paginaPrincipalaDupaAutentificare.class));
            }
            else{
                progressDialog.dismiss();
                Toast.makeText( context: PaginaAutentificare.this, "Wrong email address or password!", Toast.LENGTH_SHORT).show();
            }
        }
    });
}
```

Figura 47-Exemplu cod java pentru autentificare

- Spre deosebire de clasa java specifică activității de autentificare, cea de înregistrare nu are funcția de semnare a datelor, ci are funcția de înregistrare a acestora (figura 48) și o funcție de adăugare a datelor (figura 49) în cadrul bazei de date de pe platforma Firebase. În ceea ce privește funcția de creare a unui nou utilizator, pentru instanța firebase authentication se apelează funcția de creare bazată pe adresa de email și parolă. Dacă aceasta este cu succes atunci datele introduse de user sunt trimise spre Firebase prin intermediul funcției de `sendUserData()`, iar apoi utilizatorul este redirecționat spre pagina de autentificare. În caz contrar este afișat un mesaj de eroare. Pentru funcția de transmitere a datelor se crează o instanță a `FirebaseDatabase` prin intermediul căreia se determină calea nodului părinte din baza de date unde sunt stocate datele, apoi prin determinarea uid-ului user-ului curent se crează un nod fiu în care se salvează datele introduse.

```
private void registerUser(final String inputName, final String inputPw, final String inputEmail) {  
  
    progressDialog.setMessage("Verification...");  
    progressDialog.show();  
  
    firebaseAuth.createUserWithEmailAndPassword(inputEmail, inputPw).addOnCompleteListener(task -> {  
        if(task.isSuccessful()){  
            progressDialog.dismiss();  
            sendUserData(inputName, inputEmail);  
            Toast.makeText( context: PaginaInregistrare.this, "The account was successfully created!", Toast.LENGTH_SHORT).show();  
            startActivity(new Intent( packageContext: PaginaInregistrare.this, PaginaAutentificare.class));  
        }  
        else{  
            progressDialog.dismiss();  
            Toast.makeText( context: PaginaInregistrare.this, "For this email address already exists an account!", Toast.LENGTH_SHORT).show();  
        }  
        firebaseAuth.signOut();  
    });  
}
```

Figura 48- Funcția registerUser

```
private void sendUserData(String username, String email){  
  
    FirebaseDatabase firebaseDatabase = FirebaseDatabase.getInstance();  
    DatabaseReference users = firebaseDatabase.getReference( path: "Users");  
    UserProfile user = new UserProfile(username, password: "*****", email);  
    //users.push().setValue(user);  
    FirebaseUser us=FirebaseAuth.getInstance().getCurrentUser();  
    assert us != null;  
    String currentuser=us.getId();  
    users.child(currentuser).setValue(user);  
    /*DatabaseReference dbRef;  
    dbRef=FirebaseDatabase.getInstance().getReference("user");  
    dbRef.child("nume").setValue(username);  
    dbRef.child("email").setValue(email);*/  
}
```

Figura 49- Funcția sendUserData

- Pentru activitatea care are ca și componentă un spinner a fost necesară definirea mai multor liste de string pentru a cuprinde afișarea textului în toate limbile, în funcție de alegerea lingvistică făcută anterior în aplicație se va atribui lista de string-uri specifică, iar prin funcția *function()* utilizată în cadrul acesteia se va deschide interfața specifică alegerii făcute din lista prezentată în spinner (după cum se poate observa în figura 50).

```
textView=(TextView)findViewById(R.id.TextSelectSpinner);
spinner=(Spinner) findViewById(R.id.sp);
if(textView.getText().toString().equals("Auswählen")) {
    ArrayAdapter<CharSequence> adapter = ArrayAdapter.createFromResource( context, this, R.array.spinnerarrayDE, android.R.layout.simple_spinner_item);
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spinner.setAdapter(adapter);
    spinner.setOnItemSelectedListener(new function());
}
else if(textView.getText().toString().equals("Select"))
{
    ArrayAdapter<CharSequence> adapter = ArrayAdapter.createFromResource( context, this, R.array.spinnerarray, android.R.layout.simple_spinner_item);
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spinner.setAdapter(adapter);
    spinner.setOnItemSelectedListener(new function());
}
```

Figura 50- Metoda de selecționare a listei de valori pentru spinner

- Pentru activitatea de scanare a codului de bare (figura 51), inițial am definit permisiunile de accesare a camerei în fișierul Manifest, am stabilit forma de citire a codului de bare și dimensiunile pe care le are interfața de afișare a camerei. Apoi utilizând permisiunile definite pentru cameră am realizat accesarea acesteia și utilizarea ei pentru a extrage într-o casetă text caracterele interpretate din codul de bare scanat cu ajutorul acesteia. Pentru fiecare cod scanat am realizat tranziția din activitatea curentă în cea specifică.

```
private void initialiseDetectorsAndSources() {

    //Toast.makeText(getApplicationContext(), "Barcode scanner started", Toast.LENGTH_SHORT).show();

    barcodeDetector = new BarcodeDetector.Builder( context, this)
        .setBarcodeFormats(Barcode.ALL_FORMATS)
        .build();

    cameraSource = new CameraSource.Builder( context, this, barcodeDetector)
        .setRequestedPreviewSize( 1920, 1080)
        .setAutoFocusEnabled(true) //you should add this feature
        .build();

    surfaceView.getHolder().addCallback(new SurfaceHolder.Callback() {
        @Override
        public void surfaceCreated(SurfaceHolder holder) {
            try {
                if (ActivityCompat.checkSelfPermission( context, PaginaPrincipalaScanare.this, Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED) {
                    cameraSource.start(surfaceView.getHolder());
                } else {
                    ActivityCompat.requestPermissions( activity, PaginaPrincipalaScanare.this, new
                        String[]{Manifest.permission.CAMERA}, REQUEST_CAMERA_PERMISSION);
                }
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    });
}
```

Figura 51- Cod java specific scanării codului de bare

- În ceea ce privește setarea unei alarme (figura 52), pentru obiectul specific de alegerea orei din cadrul activității, am setat un id pentru notificarea adăugată de către utilizator și am construit o notificare care urma să fie afișată. Pentru oră a trebuit să construiesc ora în funcție de timpul curent al zilei, iar apoi am realizat setarea acesteia.

```

33 public void onClick(View view)
34 {
35     EditText editText=findViewById(R.id.editText);
36     TimePicker timePicker=findViewById(R.id.timePicker);
37
38     //setare id notificare + text
39     Intent intent=new Intent( packageContext: Alarma2.this, AlarmReciver.class);
40     intent.putExtra( name: "Notification Id",notificationId);
41     intent.putExtra( name: "todo",editText.getText().toString());
42
43     PendingIntent alarmIntent =PendingIntent.getBroadcast( context: Alarma2.this, requestCode: 0,intent, PendingIntent.FLAG_CANCEL_CURRENT);
44     AlarmManager alarmManager=(AlarmManager) getSystemService(ALARM_SERVICE);
45
46     switch (view.getId()){
47         case R.id.buttonSet:
48             int hour=timePicker.getCurrentHour();
49             int minute=timePicker.getCurrentMinute();
50
51             //create ora alarm
52             Calendar startTime=Calendar.getInstance();
53             startTime.set(Calendar.HOUR_OF_DAY, hour);
54             startTime.set(Calendar.MINUTE, minute);
55             startTime.set(Calendar.SECOND, 0);
56             long alarmStartTime=startTime.getTimeInMillis();
57
58             //setare alarma
59             alarmManager.set(AlarmManager.RTC_WAKEUP,alarmStartTime,alarmIntent);
60             break;
61
62         case R.id.buttonCancel:
63             alarmManager.cancel(alarmIntent);
64             break;
65     }
66 }

```

Figura 52-Codul java specific pentru setarea orei și a notificării specifice alarmei adăugate de către utilizator

- Pentru codul Java specific funcționalității de chatbot (figura 53), am utilizat fișierele XML create pentru răspunsul robotului, dar și pentru cel al utilizatorului. În funcție de mesajul scris de către utilizator se apelează funcția de răspuns în care se apelează API-ul online al chatbot-ului și se încarcă răspunsul definit în fișierul AIML (fișierul dedicat inteligenței artificiale care stă la baza creării chatbot-ului online) specific tag-ului care conține textul introdus de către utilizator.

```

private void getResponse(String message) {
    chatModelArrayList.add(new ChatModel(message, USER_KEY));
    chatAdapter.notifyDataSetChanged();
    String url="http://api.brainshop.ai/get?bid=100107&key=9d4v8W7c3ry8r5Gidc[wid&msg="+message;
    String BASE_URL = "http://api.brainshop.ai/";
    Retrofit retrofit=new Retrofit.Builder()
        .baseUrl(BASE_URL)
        .addConverterFactory(GsonConverterFactory.create())
        .build();
    RetrofitFitApi retrofitFitApi=retrofit.create(RetrofitFitApi.class);
    Call<MsgModel> call=retrofitFitApi.getMessage(url);
    call.enqueue(new Callback<MsgModel>() {
        @Override
        public void onResponse(Call<MsgModel> call, Response<MsgModel> response) {
            if (response.isSuccessful()){
                MsgModel msgModel=response.body();
                chatModelArrayList.add(new ChatModel(msgModel.getCont(), BOT_KEY));
                chatAdapter.notifyDataSetChanged();
                recyclerView.scrollToPosition(chatModelArrayList.size()-1);
            }
        }
        @Override
        public void onFailure(Call<MsgModel> call, Throwable t) {
            chatModelArrayList.add(new ChatModel( message "No response", BOT_KEY));
            chatAdapter.notifyDataSetChanged();
        }
    });
}

```

Figura 53-Codul java pentru funcția de chatbot

CAPITOLUL 6: UTILIZAREA SISTEMULUI

În cadrul acestui capitol am să prezint modul de utilizare al aplicației mobile, MedicalFriend, dezvoltate.

În momentul în care aplicația este lansată în acțiune, o interfață simplă este afișată, iar dacă utilizatorul așteaptă două sau trei secunde, meniul principal al aplicației o să fie afișat.

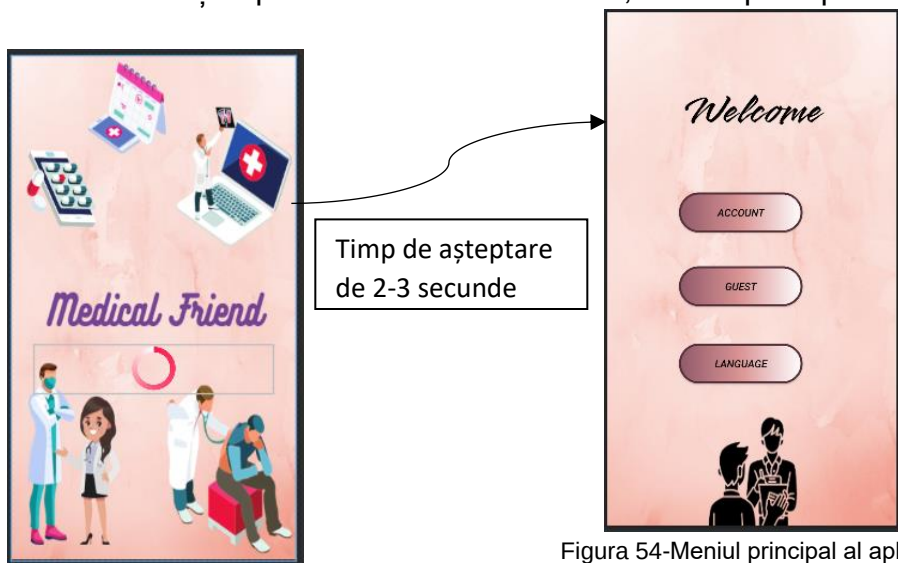


Figura 55- Interfața de deschidere a aplicației

Figura 54-Meniul principal al aplicației

În momentul în care utilizatorul dorește să schimbe limba de afișare va accesa interfața prin intermediul butonului *Language* din figura 54, iar apoi următoarea interfață se va deschide (figura 56).



Figura 56- Interfața specifică alegerii limbii de afișare

În funcție de steagul pe care se apasă se va seta limba sistemului în urma apăsării butonului, iar apoi interfața principală (figura 54) va fi afișată, de unde utilizatorul poate alege o altă opțiune. Odată ajuns în pagina care prezintă funcționalitățile (figura 54), dacă utilizatorul

optează spre funcția specifică user-ilor de tip oaspete (adică toți user-ii aplicației), adică funcțiile care nu necesită autentificare.

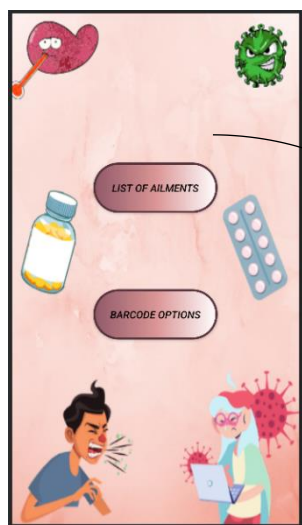


Figura 59-Interfața cu funcționalitățile destinate utilizatorilor de tip oaspete

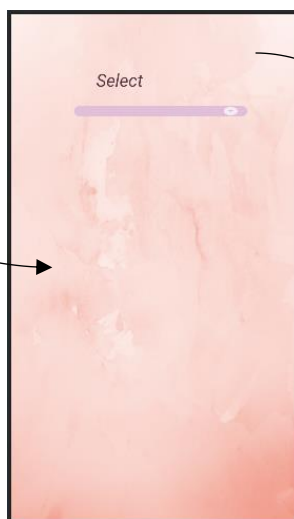


Figura 57-Interfață principală pentru listarea afecțiunilor

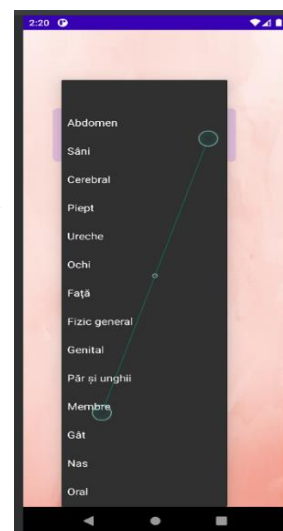


Figura 58-Valorile afișate cu ajutorul spinner-ului

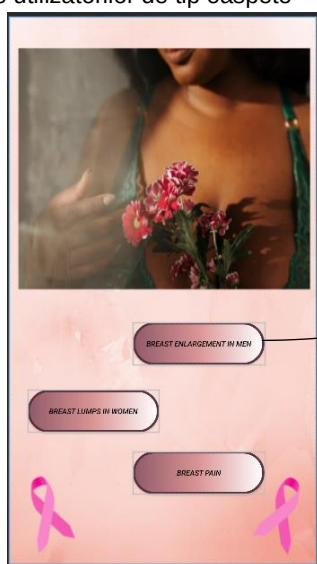


Figura 61-Interfața principală specifică afecțiunilor întâlnite la sâni

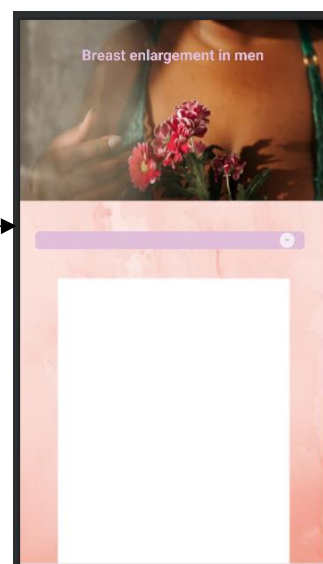


Figura 60-Interfață specifică afișării de informații

În cazul în care utilizatorul optează pentru funcția de afișare a afecțiunilor (procesul este exemplificat vizual prin figurile 57, 58, 60 și 61), acesta apasă butonul destinat listei de afecțiuni, iar în următoarea interfață dintre variantele puse la dispoziția acestuia de către spinner, alegerea va deschide o nouă interfață care conține toate afecțiunile specifice alegerii. Pentru butoanele care conțin denumirea afecțiunilor acțiunea de apăsare a acestora va rezulta în deschiderea interfețelor specifice de afișare a informațiilor. Categoria pentru care se dorește afișat textul este selectat din cel de-al doilea spinner (figura 61), iar în cadrul interfeței, zona albă dedicată textului va fi afișat.

Dacă în momentul în care e deschisă interfața din figura 59 nu este apăsat butonul pentru lista de afecțiuni, atunci utilizatorul poate opta pentru a alege funcționalitatea dedicată scanării codului de bare (figurile 63 și 62).

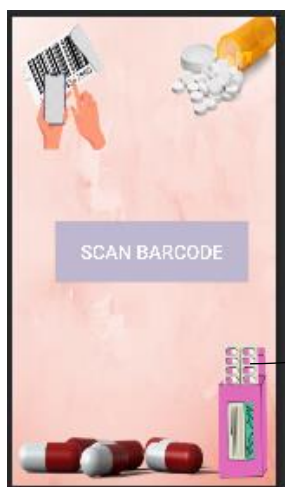


Figura 63-Interfață activității de scanare captată în Android



Figura 62-Interfață de afișare a informațiilor din prospect

Când codul este scanat datele sunt extrase în text box, iar apoi pagina specifică codului de bare este deschisă și informația este afișată din baza de date. Cu ajutorul butoanelor din fiecare interfață se poate naviga spre următoarea secțiune din prospect.

Din meniul principal din figura 54, utilizatorul poate apăsa butonul pentru contul de utilizator. Atunci acesta va fi redirecționat spre pagina care cere datele de autentificare (figura 66). După introducerea acestora și apăsarea butonului se va deschide interfața principală a utilizatorului care dispune de cont (figura 64). Dacă acesta nu are un cont, pe apăsarea textului care evidențiază acest lucru poate ajunge în pagina de creare de cont (figura 65), care după ce primește datele necesare și este apăsat butonul de creare va redirecționa utilizatorul spre pagina de autentificare pentru a valida contul creat.

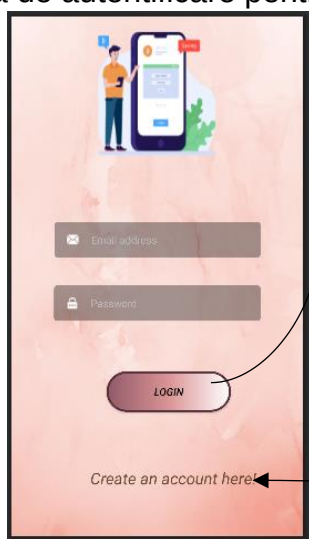


Figura 66-Interfață autentificare

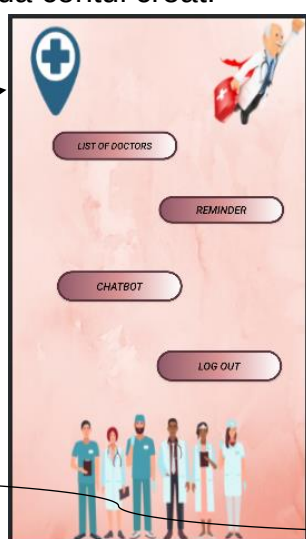


Figura 64-Interfață după autentificare

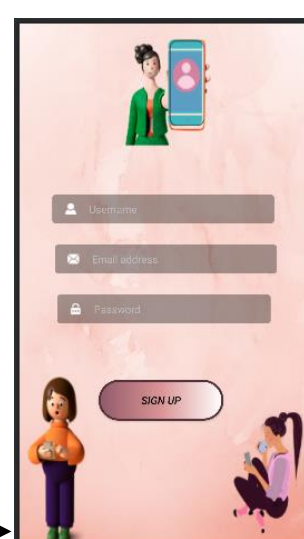


Figura 65-Interfață creare cont

Dacă după ce s-a autentificat, un utilizator, dorește să vizualizeze diferite informații despre diferite cadre medicale (figura 67), atunci prin apăsarea butonului specifi se va deschide o interfață care va afișa aceste date. Prin butoanele cu săgeți aceștia pot naviga printre informațiile afișate, fie să treacă la următoarea fie la cea precedentă.



Figura 67-Exemplu de afișare a cadrelor medicale

Dacă user-ul dorește setarea unei alarme (figura 68), atunci acesta va apăsa butonul aferent, iar în interfața deschisă prin apăsarea pe ceasul afișat se va alege ora dorită, în cadranul de notițe, acesta poate tasta informația dorită. Prin apăsarea butonului de setare alarmă este salvată, iar prin apăsarea butonului de anulare aceasta va fi ștearsă din memorie.

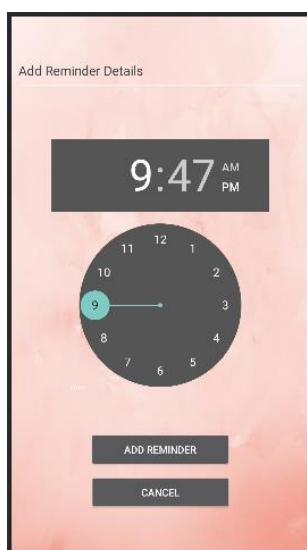


Figura 68-Interfața specifică setării de alarmă

Dacă utilizatorul dorește să poarte o discuție, în scopul destinderii, atunci acesta trebuie să apese butonul destinat chatbot-ului (figurile 69 și 70). O interfață de transfer este deschisă, iar după două sau trei secunde interfața pentru dialog este deschisă. Aici utilizatorul introduce un text în cadranul dedicat tipăririi, iar apoi prin apăsarea săgeții de transmitere a mesajului se va porni discuția cu robotul.



Figura 70-Interfață de transfer spre chatbot

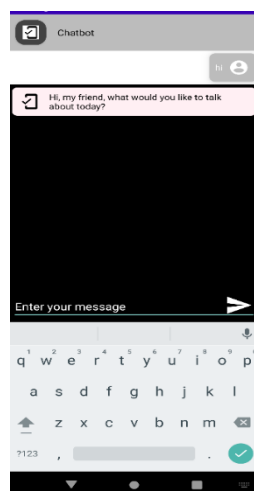


Figura 69- Interfața chatbot-ului

De asemenea dacă utilizatorul dorește să părăsească contul înainte de a închide aplicația atunci apăsând pe butonul de delogare din figura 64 acesta va fi readus în interfața principală din figura 54 și când va apăsa butonul de cont datele de autentificare îi vor fi cerute. Dacă înainte de a închide aplicația acesta nu a dorit părăsirea contului, în momentul în care aplicația va fi deschisă și butonul de cont va fi apăsat se va deschide interfața din figura 64.

CAPITOLUL 7: CONCLUZII

În concluzie pornind de la o simplă idee de a avea mai ușor acces la diferite informații din domeniul medical s-a realizat dezvoltarea proiectului, MedicalFriend. Această aplicație a fost gândită ca un mic ajutor pentru toate categoriile de persoane, astfel s-a dorit implementarea unui design și funcționare simplă și intuitivă.

Pentru un utilizator cu diferite cunoștințe în domeniul IT probabil este mult mai ușor să înțeleagă modul de funcționare al unei aplicații după puțin timp de utilizare, însă nu toate persoanele care doresc anumite informații sau funcționalități au capacitatea aceasta. Prin designul simplu aplicația ghidează utilizatorul spre toate funcționalitățile acesteia.

În ceea ce privește testarea acesteia, am utilizat emulatorul pus la dispoziție de către mediul de programare, dar și un dispozitiv fizic, pentru a observa comportamentul acesteia. Din punctul de vedere al vitezei de răspuns sau modului de afișare nu au fost observate diferențe majore sau care ar putea crea probleme în timpul utilizării.

Ca orice altă aplicație aceasta poate necesita funcționalități în plus, precum localizarea GPS al celui mai apropiat punct medical sau posibilitatea de a realiza o programare pentru un anumit cadru medical, însă acestea sunt îmbunătățiri care pot fi aduse pe viitor.

În concluzie, aplicația a fost dezvoltată să atingă principalele funcționalități descrise la începutul acestui document, printr-o interfață cât mai simplă și ușor de înțeles. Un alt lucru de menționat, acum pe final, ar fi faptul că informațiile medicale adăugate în acest proiect au fost cautate în diferite manuale medicale recomandate studenților la medicină și din prospecte ale medicamentelor reale.

CAPITOLUL 8: BIBLIOGRAFIE

- [1] Thecon, "5 avantaje ale dezvoltării de aplicații mobile pentru afacerea ta", 2018, accesat în 2022, disponibil la adresa <https://thecon.ro/5-avantaje-aplicatii-mobile/>
- [2] Wikipedia, "Sisteme de operare", 2022, accesat în 2022, disponibil la adresa https://ro.wikipedia.org/wiki/Sistem_de_operare
- [3] Mahvish Ponum, Osman Hasan, Saadia Khan, EasyDetectDisease: An Android App for Early Symptom Detection and Prevention of Childhood Infectious Diseases, Published online, 2019, <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6538239/>
- [4] Henry Hoffmann, Ada Health Our approach to assess Ada's diagnostic performance, ITU Workshop on Artificial Intelligence for Health Session 3: Data Availability and Benchmarking Geneva, Switzerland, 2018, https://www.itu.int/en/ITU-T/Workshops-and-Seminars/20180925/Documents/3_Henry%20Hoffmann.pdf
- [5] Resmi R, Shifana S, ShopEZY- An Android Application Based on Barcode Scanning and Data Analytics, IJSTE - International Journal of Science Technology & Engineering | Volume 3, 2017
- [6] Documentația Android pentru dezvoltatori, 2022, accesat în 2022, disponibil la adresa https://developer.android.com/studio/intro/?gclid=Cj0KCQjwm6KUBhC3ARIsAClwBgKN2Axex-hW_64DR5Za4eQoFIJtfODwfGvVmFowjIMozOZPjmTYZsaAi6HEALw_wcB&gclidsrc=aw.ds
- [7] James Gosling, Bill Joy, Guy Steele, Gilad Bracha, Alex Buckley, Daniel Smith, The Java® Language Specification, Java SE 11 Edition, America, September 2018, <https://cr.openjdk.java.net/~iris/se/11/latestSpec/java-se-11-jls-draft-diffs.pdf>
- [8] Wikipedia, "Java (limbaj de programare)", 2022, accesat în 2022, disponibil la adresa [https://ro.wikipedia.org/wiki/Java_\(limbaj_de_programare\)](https://ro.wikipedia.org/wiki/Java_(limbaj_de_programare))
- [9] Airman, "Introducere în Java", 2019, accesat în 2022, disponibil la adresa <https://uncoded.ro/introducere-in-java/>
- [10] W3Schools, "Java Syntax", accesat în 2022, disponibil la adresa https://www.w3schools.com/java/java_syntax.asp
- [11] Shane Conder and Lauren Darcey, "Learn Java for Android Development: Introduction to Java", 2010, accesat în 2022, disponibil la adresa <https://code.tutsplus.com/tutorials/learn-java-for-android-development-introduction-to-java--mobile-2604>
- [12] mdn web docs, "XML introduction", accesat în 2022, disponibil la adresa https://developer.mozilla.org/en-US/docs/Web/XML/XML_introduction
- [13] Peter Loshin, David Linthicum and Maxine Giza, "XML (Extensible Markup Language)", accesat în 2022, disponibil la adresa <https://www.techtarget.com/whatis/definition/XML-Extensible-Markup-Language>
- [14] Peter Loshin, David Linthicum and Maxine Giza, "XML (Extensible Markup Language)", accesat în 2022, disponibil la adresa <https://www.techtarget.com/whatis/definition/XML-Extensible-Markup-Language>
- [15] Wikipedia, "Firebase", 2022, accesat în 2022, disponibil la adresa <https://en.wikipedia.org/wiki/Firebase>

- [16] Documentație Firebase, accesat în 2022, disponibil la adresa <https://firebase.google.com/docs/>
- [17] Documentația Android pentru dezvoltatori, "Build a Responsive UI with ConstraintLayout", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/training/constraint-layout>
- [18] Documentația Android pentru dezvoltatori, "ProgressBar", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/reference/android/widget/ProgressBar>
- [19] Documentația Android pentru dezvoltatori, "Button", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/reference/android/widget/Button>
- [20] Documentația Android pentru dezvoltatori, "Relative Layout", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/guide/topics/ui/layout/relative>
- [21] Documentația Android pentru dezvoltatori, "AutoCompleteTextView", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/reference/android/widget/Auto-CompleteTextView>
- [22] Documentația Android pentru dezvoltatori, "Linear Layout", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/reference/android/widget/LinearLayout>
- [23] Documentația Android pentru dezvoltatori, "Spinner", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/guide/topics/ui/controls/spinner>
- [24] Documentația Android pentru dezvoltatori, "Surface View", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/reference/android/view/SurfaceView>
- [25] Documentația Android pentru dezvoltatori, "Time Picker", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/reference/android/widget/TimePicker>
- [26] Documentația Android pentru dezvoltatori, "Edit Text", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/reference/android/widget/EditText>
- [27] Documentația Android pentru dezvoltatori, "Image View", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/reference/android/widget/ImageView>
- [28] GeeksforGeeks, "CardView in Android With Example", 2021, accesat în 2022, disponibil la adresa <https://www.geeksforgeeks.org/cardview-in-android-with-example/>
- [29] Documentația Android pentru dezvoltatori, "Recycler View", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/jetpack/androidx/releases/recyclerview>
- [30] Documentația Android pentru dezvoltatori, "Scroll View", 2022, accesat în 2022, disponibil la adresa <https://developer.android.com/reference/android/widget/ScrollView>

Listă de figuri

Figură 1-Spațiul utilizat în cadrul Firebase Storage pentru imaginile specifice informațiilor despre medici.....	6
Figură 2-Interfața aplicației de informare EasyDetectDisease	8
Figură 3-Interfața aplicației de tip chatbot Ada	9
Figură 4-Organizarea pe module a aplicației MedicalFriend	11
Figură 5-Exemplu de definire a unui fișier XML din cadrul proiectului MedicalFriend	13
Figură 6-Schema bloc a aplicației dezvoltate.....	14
Figură 7-Captură de ecran cu activity_main.xml	16
Figură 8-Interfața specifică deschiderii aplicației.....	16
Figură 9-Interfața principală a aplicației	17
Figură 10-Captură de ecran cu codul XML specific obiectelor descrise la acest subpunct....	17
Figură 11-Exemplu de implementare a AutoCompleteTextView-ului din cadrul proiectului ...	18
Figură 12-Interfața de autentificare a aplicației	18
Figură 13- Interfața specifică funcției de creare cont pentru un utilizator.....	18
Figură 14-Exemplu de implementare a LinearLayout-ului din fișierele XML ale proiectului ...	19
Figură 15- Interfața de schimbare a limbii	19
Figură 16-Interfața dedicată tuturor utilizatorilor fără necesitatea de a crea cont.....	19
Figură 17-Codul XML specific Spinner-ului	20
Figură 18-Lista de valori afișate în urma apăsării Spinner-ului	20
Figură 19-Codul XML pentru Spinner	20
Figură 20-Interfața dedicată scanării codului de bare	20
Figură 21-Interfața pentru alegerea afecțiunilor	21
Figură 22-Interfața principală după autentificare.....	21
Figură 23-Interfața pentru setarea unei alarme.....	21
Figură 24-Design-ul din Android studio pentru interfața de listare a medicilor	22
Figură 25-Exemplu de cod XML pentru implementarea unui ImageView	22
Figură 26-Exemplu de afișare a datelor despre medici din Firebase în momentul rulării aplicației	22
Figură 27-Interfața de tranziție, captură de ecran realizată în Android Studio	23
Figură 28-Interfață principală chatbot în Android Studio	23
Figură 29-Codul XML pentru crearea design-ului bulinei de răspuns a utilizatorului cu ajutorul CardView	24
Figură 30-Imagine exemplificativă cu funcționalitatea chatbot-ului	24
Figură 31-Imagine din Android Studio cu design-ul care utilizează CardView pentru crearea răspunsului utilizatorului.....	24
Figură 32- Structura bazei de date.....	24
Figură 33- Structura nodului Medici	25
Figură 34-Structura nodului Pastile.....	25
Figură 35-Informațiile salvate în Firebase Authentication la crearea unui cont de utilizator...	26
Figură 36-Structura Firebase Storage.....	26

Figură 37-Structura nodului Users	26
Figură 38- Schema arhitecturală a sistemului dezvoltat.....	27
Figură 39- Lista cu versiunile Android lansate, în ordine cronologică	28
Figură 40- Codul funcției de transfer între două interfețe cu delay din clasa Java specifică activității principale a aplicației	28
Figură 41- Captură de ecran cu codul specific ascunderii barei de opțiuni	29
Figură 42-Cod java pentru implementarea funcționalității barei de progres	29
Figură 43-Exemplu de cod java care atribuie posibilitatea de a schimba activitățile prin apăsarea unui element de tip buton din cadrul interfeței curente.....	29
Figură 44-Funcția java de setare a limbii de afișare a aplicației.....	30
Figură 45-Funcția de inițializare din clasa de autentificare	30
Figură 46-Cod java al funcției de autentificare utilizator cu Firebase	31
Figură 47-Exemplu cod java pentru autentificare	31
Figură 48- Funcția registerUser	32
Figură 49- Funcția sendUserData	32
Figură 50- Metoda de selecționare a listei de valori pentru spinner	33
Figură 51- Cod java specific scanării codului de bare	33
Figură 52-Codul java specific pentru setarea orei și a notificării specifice alarmei adăugate de către utilizator.....	34
Figură 53-Codul java pentru funcția de chatbot	34
Figură 54-Meniul principal al aplicației	35
Figură 55- Interfața de deschidere a aplicației	35
Figură 56- Interfața specifică alegerii limbii de afișare	35
Figură 57-Valorile afișate cu ajutorul spinner-ului	36
Figură 58-Interfață principală pentru listarea afecțiunilor	36
Figură 59-Interfața cu funcționalitățile destinate utilizatorilor de tip oaspete	36
Figură 60-Interfață specifică afișării de informații.....	36
Figură 61-Interfața principală specifică afecțiunilor întâlnite la sâni	36
Figură 62-Interfață de afișare a informațiilor din prospect.....	37
Figură 63-Interfață activității de scanare captată în Android	37
Figură 66-Interfață autentificare	37
Figură 64-Interfață după autentificare	37
Figură 65-Interfață creare cont.....	37
Figură 67-Exemplu de afișare a cadrelor medicale.....	38
Figură 68-Interfața specifică setării de alarmă	38
Figură 70- Interfața chatbot-ului	39
Figură 69-Interfață de transfer spre chatbot.....	39

Listă de tabele

Tabel 1- Tabel cu comparațiile între aplicațiile prezentate și aplicația dezvoltată	10
---	----